# 18
# 트랜스포머

RNN 없이 어텐션만으로 만든 모델입니다. BERT와 GPT가 모두 이 구조에서 나왔습니다. 문장 "I am a student"가 "je suis étudiant"가 되기까지, 부품을 하나씩 계산해 보고 마지막에 작은 트랜스포머를 직접 조립해 번역시킵니다.

### 오늘 다루는 것 (18-01 트랜스포머 완전 정복)

1. 큰 그림: 왜 RNN을 버렸나, 전체 구조와 하이퍼파라미터
2. 포지셔널 인코딩: 단어의 순서를 알려 주는 방법
3. 셀프 어텐션: Q, K, V와 스케일드 닷-프로덕트 어텐션
4. 멀티 헤드 어텐션과 패딩 마스크
5. 인코더 층 완성: FFNN, 잔차 연결, 층 정규화
6. 디코더: 룩-어헤드 마스크, 인코더-디코더 어텐션, 번역

### 끝나고 나면 할 수 있는 것

1. $\text{softmax}(QK^T/\sqrt{d_k})V$를 단어 하나에 대해 손으로 따라갈 수 있다.
2. 인코더 층과 디코더 층의 서브층을 순서대로 말하고, 텐서의 모양이 유지되는 이유를 설명할 수 있다.
3. 트랜스포머의 세 가지 어텐션과 두 가지 마스크가 각각 어디에 왜 쓰이는지 설명할 수 있다.

2026년 10월 15일 목요일 14:30

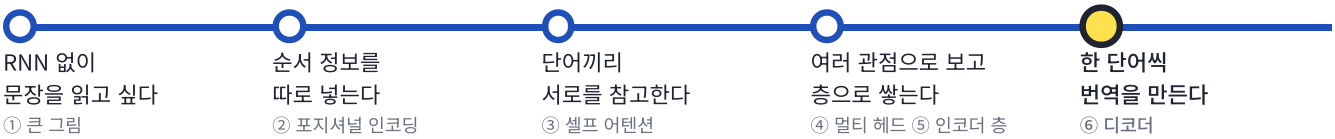
실습은 모두 코랩 기본 환경에서 실행됩니다(설치 없음). 실습 6을 파일로 저장한 뒤 실습 7을 실행합니다.

# 18

# 트랜스포머 (Transformer)

2017년 구글의 논문 "Attention is all you need"가 내놓은 모델입니다. 제목 그대로 어텐션만으로 인코더와 디코더를 만들었고, RNN보다 번역을 더 잘했습니다. 교재는 이 내용을 한 소챕터(18-01)에 담았습니다. 분량이 많아 이 자료는 여섯 부분으로 나눠, 입력에서 출력까지 데이터가 지나가는 순서대로 따라갑니다.

## 왜 배우나: 오늘의 학습 경로




장 전체에서 하나의 상황을 씁니다. 영어 문장 "I am a student"를 프랑스어 "je suis étudiant"로 번역하는 일입니다(교재의 예문). 개념마다 정의와 식을 먼저 세우고, 이 문장에 대입해 숫자로 확인합니다. 그림과 실습의 벡터는 손으로 따라갈 수 있게 4칸, 2칸으로 줄였습니다. 논문의 실제 크기는 512칸, 64칸입니다.

**먼저 알아 둘 것.** 이 장은 16장(seq2seq)과 17장(어텐션)을 바탕으로 합니다. 두 장의 내용 가운데 여기서 필요한 것은 ①부와 ③부의 정의에서 짧게 다시 설명합니다.

# 18-01 ① 큰 그림: 왜 RNN을 버렸나

핵심 한 줄

**트랜스포머는 인코더-디코더 구조는 그대로 두고, 단어를 차례로 읽는 RNN 대신 모든 단어가 서로를 직접 참고하는 어텐션으로 층을 만들어 여러 개 쌓은 모델이다.**

왜 배우나

뒤의 다섯 부분은 부품 하나하나를 들여다봅니다. 그 전에 전체 지도를 봐 두어야 각 부품이 어디에 끼워지는지 알 수 있습니다. 트랜스포머가 무엇을 그대로 두고 무엇을 바꿨는지, 그리고 모양을 정하는 숫자 네 개를 먼저 정리합니다.

## 1) seq2seq의 한계에서 트랜스포머로

정의

- 인코더-디코더 구조: 인코더(encoder)가 입력 문장을 읽어 벡터로 바꾸고, 디코더(decoder)가 그 벡터를 보며 출력 문장을 한 단어씩 만드는 구조. 번역, 요약처럼 문장을 넣어 문장을 얻는 문제에 씁니다(16장).
- seq2seq: 인코더와 디코더를 각각 RNN(LSTM, GRU)으로 만든 모델. 인코더의 마지막 은닉 상태 하나에 입력 문장 전체를 담아 디코더에 넘깁니다.
- 어텐션(attention): 디코더가 단어를 만들 때마다 입력 문장의 모든 단어를 다시 보고, 지금 필요한 단어에 더 큰 비중을 두어 참고하는 방법(17장).

**RNN으로 만든 seq2seq의 문제.**

- **정보 손실**: 긴 문장을 벡터 하나에 눌러 담으면 앞쪽 내용이 흐려집니다. 어텐션은 이것을 보완하려고 나온 방법입니다.
- **먼 단어끼리의 거리**: RNN에서 첫 단어의 정보가 열 번째 단어에 닿으려면 셀을 9번 지나야 합니다. 지날 때마다 흐려집니다(07-02의 장기 의존성 문제).
- **느린 계산**: 앞 시점의 계산이 끝나야 다음 시점을 계산할 수 있어, 한 문장 안의 단어들을 동시에 계산하지 못합니다.

**트랜스포머의 생각.** 어텐션은 "어떤 두 단어든 직접 연결한다"는 성질이 있습니다. 그렇다면 RNN을 보완하는 데 쓰지 말고, **RNN을 빼고 어텐션만으로** 인코더와 디코더를 만들면 됩니다. 어떤 두 단어도 한 걸음에 닿고, 모든 단어를 한꺼번에 계산할 수 있습니다.

| | RNN으로 만든 seq2seq | 트랜스포머 |
|---|---|---|
| 문장을 읽는 방법 | 한 단어씩 차례로 | 모든 단어를 한꺼번에 |
| 멀리 있는 두 단어 | 사이의 셀을 모두 지나야 닿음 | 어텐션으로 한 걸음에 닿음 |
| 단어의 순서를 아는 방법 | 읽는 순서가 곧 순서 | 따로 알려 줘야 함 (포지셔널 인코딩, ②부) |
| 한 문장 안의 병렬 계산 | 불가능 | 가능 |
| 그대로인 것 | 인코더-디코더 구조, 디코더가 <sos>에서 시작해 <eos>가 나올 때까지 한 단어씩 만드는 방식 | |

예시 적용

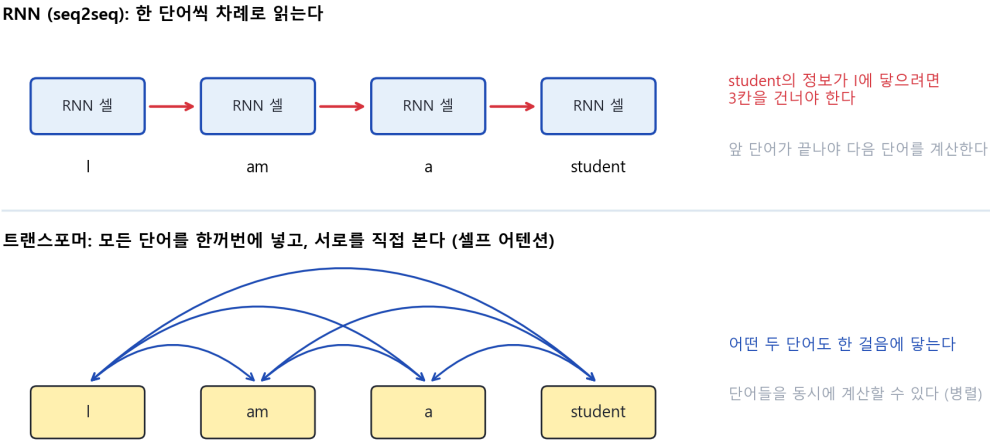



그림 1 "I am a student"를 읽는 두 방법. RNN은 차례로 넘겨 주고, 트랜스포머는 모든 단어가 서로를 직접 본다

RNN에서 student의 정보가 I의 자리에 영향을 주려면(양방향이라 해도) 셀 3개를 건너야 합니다. 단어가 4개일 때는 문제가 없지만 50개라면 49개를 건넙니다. 트랜스포머에서는 문장이 아무리 길어도 두 단어 사이는 항상 한 걸음입니다.

## 2) 전체 구조와 하이퍼파라미터

정의

트랜스포머는 같은 구조의 인코더 층을 여러 개 쌓고, 같은 구조의 디코더 층을 여러 개 쌓습니다. seq2seq에서는 RNN 하나가 여러 시점을 가졌다면, 트랜스포머에서는 층이 여러 개입니다.

- **인코더 층**의 서브층(sublayer) 2개: 멀티 헤드 셀프 어텐션(③④부) → 피드 포워드 신경망(⑤부)
- **디코더 층**의 서브층 3개: 마스크드 멀티 헤드 셀프 어텐션 → 인코더-디코더 어텐션 → 피드 포워드 신경망(⑥부)
- 입력은 인코더와 디코더 모두 **임베딩 + 포지셔널 인코딩**(②부)을 거쳐 들어갑니다.
- 인코더의 **마지막 층의 출력**이 디코더의 모든 층에 전달됩니다.

모델의 크기는 하이퍼파라미터(사용자가 정하는 값) 네 개로 정해집니다.

| 기호 | 뜻 | 논문의 값 | 이 자료의 그림 |
|---|---|---|---|
| $d_{model}$ | 단어 하나를 나타내는 벡터의 크기. 임베딩의 크기이자, **모든 층의 입력과 출력의 크기** | 512 | 4 |
| `num_layers` | 인코더 층과 디코더 층을 각각 몇 개 쌓는가 | 6 | |
| `num_heads` | 어텐션을 몇 개로 나눠 동시에 하는가 (④부) | 8 | 2 |
| $d_{ff}$ | 피드 포워드 신경망의 은닉층의 크기 (⑤부) | 2048 | 16 |

이 장에서 가장 자주 나오는 모양은 **(seq_len, $d_{model}$)**입니다. seq_len은 문장의 단어 수입니다. 문장 하나는 "단어 수만큼의 행, 행마다 $d_{model}$칸"인 행렬이고, 인코더와 디코더의 모든 층은 이 모양을 받아 같은 모양을 내보냅니다. 그래서 같은 층을 몇 개든 쌓을 수 있습니다.

예시 적용

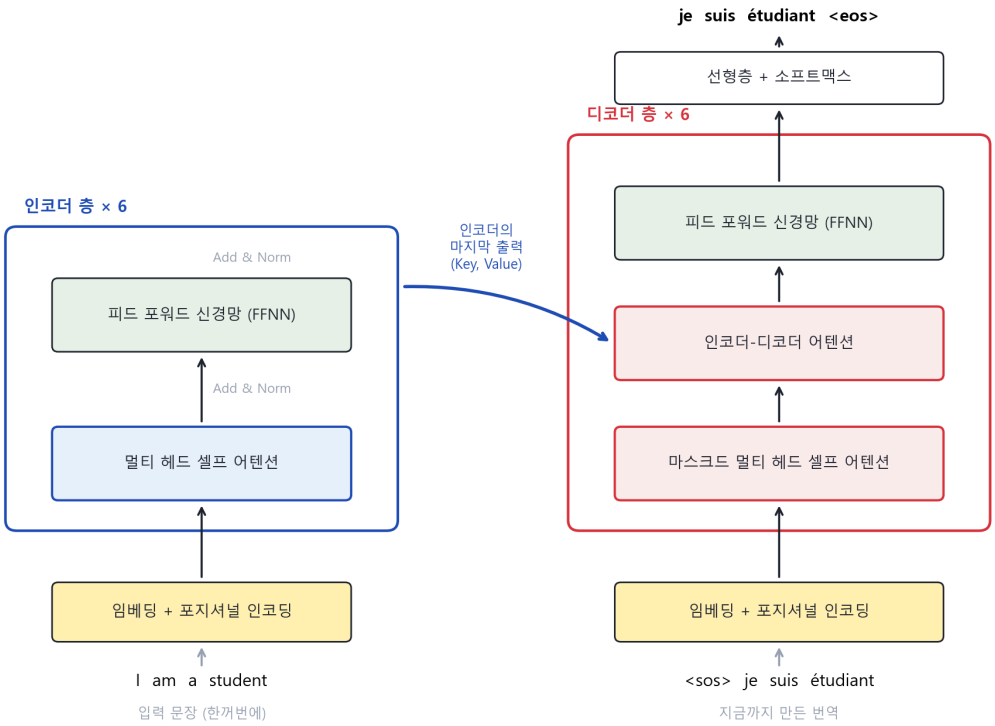



그림 2 "I am a student"를 번역하는 트랜스포머의 전체 구조. 왼쪽 인코더가 입력 문장을 읽고, 오른쪽 디코더가 지금까지 만든 번역과 인코더의 출력을 보고 다음 단어를 낸다

- 인코더에는 "I am a student" 네 단어가 **한꺼번에** 들어갑니다. 논문 크기라면 모양은 (4, 512)이고, 6개의 층을 지나도 (4, 512)입니다.
- 디코더에는 <sos>(시작 표시)와 지금까지 만든 단어들이 들어가고, 다음 단어를 냅니다. <eos>(끝 표시)가 나오면 멈춥니다.
- 파란 화살표가 인코더와 디코더를 잇는 유일한 통로입니다. 디코더는 이 통로로 입력 문장을 참고합니다.

비유

### 귓속말 전달과 원탁 회의

귓속말 전달 놀이에서는 옆 사람에게만 말할 수 있어서, 첫 사람의 말이 마지막 사람에게 닿을 때쯤이면 내용이 바뀌어 있습니다. 앞사람이 말을 마쳐야 다음 사람이 말할 수 있어 시간도 오래 걸립니다. 원탁 회의에서는 모두가 서로의 말을 직접 듣습니다. 누구의 말이든 한 번에 닿고, 각자 동시에 메모할 수 있습니다. 대신 누가 어느 자리에 앉았는지는 이름표가 없으면 알 수 없습니다.

**귓속말 전달** = = RNN (차례로 넘겨 줌)
**원탁 회의** = = 트랜스포머 (서로를 직접 참고)
**전달하다 바뀌는 말** = = 장기 의존성 문제
**자리 이름표** = = 포지셔널 인코딩 (②부)

## ① 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

**1** 트랜스포머는 인코더-디코더 구조를 쓰지 않는다. (O / X) 이름 _____

**2** 트랜스포머의 인코더는 문장의 단어를 한꺼번에 입력받는다. (O / X) 이름 _____

**3** 트랜스포머가 RNN 대신 쓰는 것은? 이름 _____

① 합성곱 ② 어텐션 ③ 원-핫 인코딩

**4** 인코더 층 하나에 들어 있는 서브층 2개는? 이름 _____

① 셀프 어텐션, 피드 포워드 신경망 ② 임베딩, 소프트맥스 ③ LSTM, GRU

**5** 논문에서 단어 벡터의 크기 $d_{model}$은 _____이다. 이름 _____

**6** 인코더의 모든 층은 (seq_len, _____) 모양을 받아 같은 모양을 내보낸다. 이름 _____

**7** RNN으로 만든 seq2seq의 문제 두 가지를 말하고, 귓속말 전달과 원탁 회의에 빗대어 설명해 보세요. 이름 _____

# 18-01 ② 포지셔널 인코딩

핵심 한 줄

**트랜스포머는 단어를 한꺼번에 읽어 순서를 모르므로, 위치마다 정해진 사인·코사인 값을 임베딩 벡터에 더해 "몇 번째 단어인지"를 알려 준다.**

왜 배우나

RNN을 뺀 대가가 여기서 나옵니다. RNN은 단어를 차례로 읽으니 순서를 저절로 알았습니다. 트랜스포머에게 "I am a student"와 "student a am I"는 같은 단어 묶음입니다. 뜻이 순서에 달려 있으므로("개가 사람을 물었다"와 "사람이 개를 물었다") 순서를 알려 주는 장치가 꼭 필요합니다.

## 1) 위치 정보를 더한다

정의

- 포지셔널 인코딩(positional encoding): 단어의 위치를 나타내는 벡터. 임베딩 벡터와 크기가 같고($d_{model}$), 임베딩 벡터에 **더해서** 모델에 넣습니다.
- 문장 전체로 보면 행렬의 덧셈입니다. 임베딩 벡터를 모은 문장 행렬(seq_len, $d_{model}$)에 같은 모양의 포지셔널 인코딩 행렬을 더합니다. 모양은 바뀌지 않습니다.

$$PE_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \qquad PE_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

기호를 하나씩 읽으면 이렇습니다.

- $pos$: 문장에서 단어의 위치. 첫 단어가 0, 둘째가 1입니다. 행렬의 **행 번호**입니다.
- $2i$, $2i+1$: 벡터 안에서의 칸 번호. 행렬의 **열 번호**입니다. 짝수 칸(0, 2, 4, …)에는 사인을, 홀수 칸(1, 3, 5, …)에는 코사인을 씁니다. 0번 칸과 1번 칸이 $i=0$으로 짝을 이루고, 2번과 3번이 $i=1$로 짝을 이룹니다.
- $d_{model}$: 벡터의 크기.
- $10000^{2i/d_{model}}$: 칸마다 다른 "느리게 하는 정도". 앞쪽 칸($i$가 작음)은 1에 가까워 위치가 바뀔 때마다 값이 빠르게 변하고, 뒤쪽 칸은 값이 커서 아주 느리게 변합니다.

**왜 사인과 코사인일까?**

- 값이 항상 -1과 1 사이라서, 문장이 길어져도 임베딩 값을 압도하지 않습니다(위치 번호 0, 1, 2, …, 99를 그대로 더하면 뒤쪽 단어는 위치 값에 묻힙니다).
- 빠르게 변하는 칸과 느리게 변하는 칸이 섞여 있어, 위치마다 벡터 전체의 무늬가 서로 다릅니다. 시계의 초침, 분침, 시침을 함께 보면 시각을 하나로 정할 수 있는 것과 같습니다.
- 학습할 것이 없는 정해진 식이라서, 학습 때 보지 못한 긴 문장의 위치도 계산할 수 있습니다.

예시 적용

"I am a student"(단어 4개), $d_{model}=4$로 계산합니다(실습 1). 둘째 단어 am($pos=1$)의 행은 이렇게 나옵니다.

| 칸 | $i$ | 식 | 값 |
|---|---|---|---|
| 0 | 0 | $\sin(1/10000^{0/4}) = \sin(1)$ | 0.841 |
| 1 | 0 | $\cos(1/10000^{0/4}) = \cos(1)$ | 0.540 |
| 2 | 1 | $\sin(1/10000^{2/4}) = \sin(0.01)$ | 0.010 |
| 3 | 1 | $\cos(1/10000^{2/4}) = \cos(0.01)$ | 1.000 |

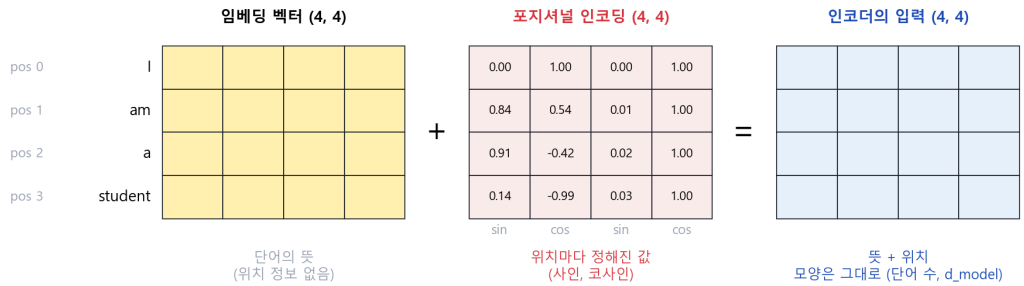



그림 3 임베딩 벡터의 행렬에 포지셔널 인코딩 행렬을 더한다. 앞의 두 칸은 위치마다 크게 바뀌고, 뒤의 두 칸은 0.01씩 천천히 바뀐다

- **같은 단어, 다른 위치.** "i am what i am"에서 0번째 i와 3번째 i의 임베딩 벡터는 같습니다(True). 포지셔널 인코딩을 더한 뒤에는 [1.54, 0.71, -2.18, 1.57]과 [1.68, -1.28, -2.15, 1.57]로 달라집니다(False). 모델은 이 차이로 위치를 구분합니다.
- **가까운 위치는 비슷한 값.** 교재의 크기(단어 50개, 128차원)로 만든 행렬에서 위치 10의 벡터와의 코사인 유사도는 위치 11이 0.970, 위치 15가 0.737, 위치 30이 0.608입니다. 멀어질수록 덜 비슷합니다.

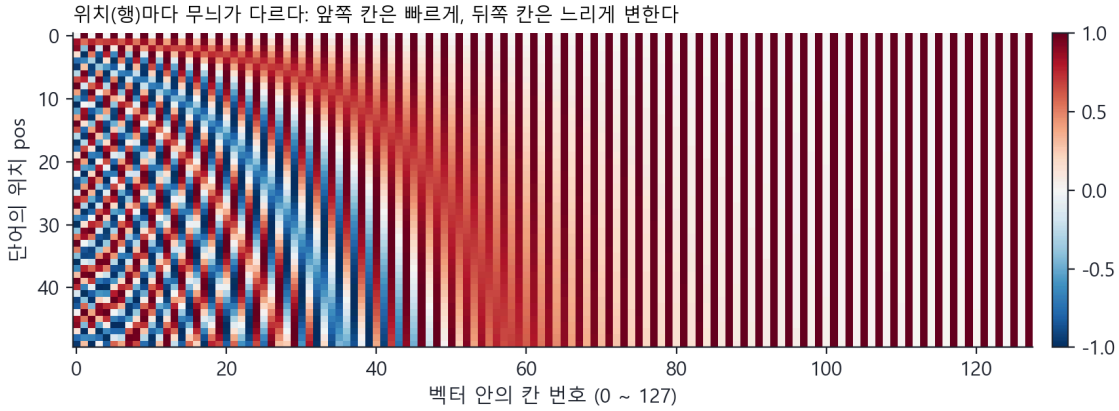



그림 4 50 × 128 포지셔널 인코딩 행렬(실습 1의 실제 값). 한 행이 한 위치의 벡터다. 빨강은 1, 파랑은 -1

비유

### 섞인 원고에 찍힌 쪽 번호

원고를 한 장씩 차례로 건네받으면 순서를 따로 적을 필요가 없습니다. 하지만 원고 뭉치를 한꺼번에 책상에 펼쳐 놓으면 어느 장이 먼저인지 알 수 없습니다. 그래서 장마다 쪽 번호를 찍어 둡니다. 내용은 그대로이고, 종이마다 번호가 더해졌을 뿐입니다.

| | |
|---|---|
| 차례로 건네받기 | = = RNN |
| 한꺼번에 펼쳐 놓기 | = = 트랜스포머의 입력 |
| 원고의 내용 | = = 임베딩 벡터 |
| 쪽 번호 도장 | = = 포지셔널 인코딩 (더하기) |

실습 1

## 포지셔널 인코딩을 식 그대로 만들기

```python
import math

import torch

def positional_encoding(max_len, d_model):
    pos = torch.arange(max_len).float().unsqueeze(1)   # (max_len, 1): 단어의 위치
    two_i = torch.arange(0, d_model, 2).float()        # 0, 2, 4, ... = 식의 2i
    angle = pos / 10000 ** (two_i / d_model)           # (max_len, d_model / 2)
    pe = torch.zeros(max_len, d_model)
    pe[:, 0::2] = torch.sin(angle)                     # 짝수 칸: 사인
    pe[:, 1::2] = torch.cos(angle)                     # 홀수 칸: 코사인
```

```
    return pe

# 1) 단어 4개, d_model = 4인 포지셔널 인코딩 행렬
pe = positional_encoding(4, 4)
print("포지셔널 인코딩 행렬:", tuple(pe.shape), "= (단어 수, d_model)")
for pos, row in enumerate(pe):
    print(f"  pos {pos}:", [f"{v:.3f}" for v in row.tolist()])

# 2) 식으로 직접 확인: pos = 1인 행
hand = [math.sin(1 / 10000 ** (0 / 4)), math.cos(1 / 10000 ** (0 / 4)),
        math.sin(1 / 10000 ** (2 / 4)), math.cos(1 / 10000 ** (2 / 4))]
print("pos 1을 식으로 계산:", [f"{v:.3f}" for v in hand])

# 3) 같은 단어라도 위치가 다르면 모델에 들어가는 벡터가 달라진다
torch.manual_seed(0)
vocab = {"i": 0, "am": 1, "what": 2}
embedding = torch.nn.Embedding(len(vocab), 4)
words = ["i", "am", "what", "i", "am"]
ids = torch.tensor([vocab[w] for w in words])
emb = embedding(ids).detach()                          # (5, 4)
x = emb + positional_encoding(5, 4)                    # 임베딩 + 위치
print("\n임베딩만: 0번째 i = 3번째 i ?", torch.equal(emb[0], emb[3]))
print("위치를 더한 뒤: 0번째 i = 3번째 i ?", torch.equal(x[0], x[3]))
print("  0번째 i:", [f"{v:.2f}" for v in x[0].tolist()])
print("  3번째 i:", [f"{v:.2f}" for v in x[3].tolist()])

# 4) 교재의 크기: 단어 50개, 128차원. 가까운 위치일수록 값이 비슷하다
big = positional_encoding(50, 128)
print("\n큰 행렬:", tuple(big.shape), " 값의 범위:",
      f"{big.min():.2f} ~ {big.max():.2f}")
for other in [11, 15, 30]:
    sim = torch.cosine_similarity(big[10], big[other], dim=0)
    print(f"  위치 10과 위치 {other}의 코사인 유사도: {sim:.3f}")
torch.save(big, "pe_50x128.pt")
```

실행 결과

```
포지셔널 인코딩 행렬: (4, 4) = (단어 수, d_model)
  pos 0: ['0.000', '1.000', '0.000', '1.000']
  pos 1: ['0.841', '0.540', '0.010', '1.000']
  pos 2: ['0.909', '-0.416', '0.020', '1.000']
  pos 3: ['0.141', '-0.990', '0.030', '1.000']
pos 1을 식으로 계산: ['0.841', '0.540', '0.010', '1.000']

임베딩만: 0번째 i = 3번째 i ? True
위치를 더한 뒤: 0번째 i = 3번째 i ? False
  0번째 i: ['1.54', '0.71', '-2.18', '1.57']
  3번째 i: ['1.68', '-1.28', '-2.15', '1.57']

큰 행렬: (50, 128)  값의 범위: -1.00 ~ 1.00
  위치 10과 위치 11의 코사인 유사도: 0.970
  위치 10과 위치 15의 코사인 유사도: 0.737
  위치 10과 위치 30의 코사인 유사도: 0.608
```

직접 해 보기 `positional_encoding(4, 4)` 의 3번 칸은 네 위치 모두 1.000으로 보입니다. 정말 같은 값일까요? `positional_encoding(1000, 4)[999]` 를 출력해 뒤쪽 칸이 느리게 변한다는 것을 확인해 보세요.

## ② 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

**1** 포지셔널 인코딩은 임베딩 벡터에 이어 붙여서(concat) 벡터를 길게 만든다. (O / X) 이름 _____

**2** 포지셔널 인코딩의 값은 학습으로 정해지는 가중치다. (O / X) 이름 _____

**3** 트랜스포머에 포지셔널 인코딩이 필요한 이유는? 이름 _____

① 단어를 한꺼번에 읽어 순서를 모르기 때문 ② 단어 집합이 커서 ③ 계산을 빠르게 하려고

**4** 식에서 $pos$가 뜻하는 것은? 이름 _____

① 벡터 안의 칸 번호 ② 문장에서 단어의 위치 ③ 단어 집합에서의 번호

**5** 벡터의 짝수 칸에는 사인 함수를, 홀수 칸에는 _____ 함수를 쓴다. 이름 _____

**6** 포지셔널 인코딩의 값은 항상 _____과 1 사이다. 이름 _____

**7** 포지셔널 인코딩이 무엇인지 정의하고, 섞인 원고의 쪽 번호에 빗대어 설명해 보세요. 이름 _____

# 18-01 ③ 셀프 어텐션

핵심 한 줄

**셀프 어텐션은 문장의 각 단어가 같은 문장의 모든 단어와 점수를 내어 비율을 정하고, 그 비율대로 다른 단어들의 정보를 섞어 "문맥이 반영된 자기 벡터"를 만드는 연산이다.**

왜 배우나

트랜스포머의 심장입니다. RNN이 하던 일, 곧 "주변 단어를 보고 이 단어의 뜻을 문맥에 맞게 고치는 일"을 셀프 어텐션이 대신합니다. 논문의 식 $\mathrm{softmax}(QK^T/\sqrt{d_k})V$ 한 줄이 전부이고, 이 식을 단어 하나로 따라가면 나머지 부품은 모두 이것의 변형입니다.

## 1) 어텐션 복습과 셀프 어텐션

정의

어텐션은 세 가지 역할로 이루어집니다(17장).

- 쿼리(Query, Q): 질문하는 쪽. "지금 나에게 필요한 정보가 무엇인가"
- 키(Key, K): 질문과 맞춰 볼 대상의 이름표. 쿼리와 키가 얼마나 맞는지 점수를 냅니다.
- 값(Value, V): 실제로 가져올 내용. 키마다 값이 하나씩 짝지어 있습니다.

$$\text{Query} \xrightarrow{\text{vs all Keys}} \text{scores} \xrightarrow{\text{softmax}} \text{weights (sum = 1)} \xrightarrow{\text{weighted sum of Values}} \text{attention value}$$

쿼리 하나로 모든 키와 점수(어텐션 스코어)를 내고, 소프트맥스로 합이 1인 비율(어텐션 분포)로 바꾼 뒤, 그 비율대로 값들을 섞습니다(가중합). 결과가 어텐션 값입니다.

| | seq2seq의 어텐션 (17장) | 셀프 어텐션 |
|---|---|---|
| Q | 디코더의 은닉 상태 | 입력 문장의 모든 단어 벡터 |
| K, V | 인코더의 은닉 상태들 | 입력 문장의 모든 단어 벡터 |
| 누가 누구를 보나 | 출력 문장의 단어가 입력 문장을 | 문장의 단어가 **같은 문장**의 단어들을 |

- 셀프 어텐션(self-attention): Q, K, V가 모두 같은 문장에서 나오는 어텐션. 문장이 자기 자신을 참고합니다.
- "같다"는 것은 **출처**가 같다는 뜻입니다. 벡터의 값이 같다는 뜻이 아닙니다. 아래에서 보듯 같은 단어 벡터에 서로 다른 가중치 행렬을 곱해 Q, K, V를 따로 만듭니다.

예시 적용

"The animal didn't cross the street because it was too tired"(그 동물은 길을 건너지 않았다. 너무 피곤했기 때문이다)에서 it은 animal일까요, street일까요? 사람은 피곤한 것이 동물이라는 것을 압니다. 셀프 어텐션에서 it이 쿼리가 되어 문장의 모든 단어와 점수를 내면, animal의 비율이 높게 나와 it의 벡터에 animal의 정보가 많이 섞입니다.

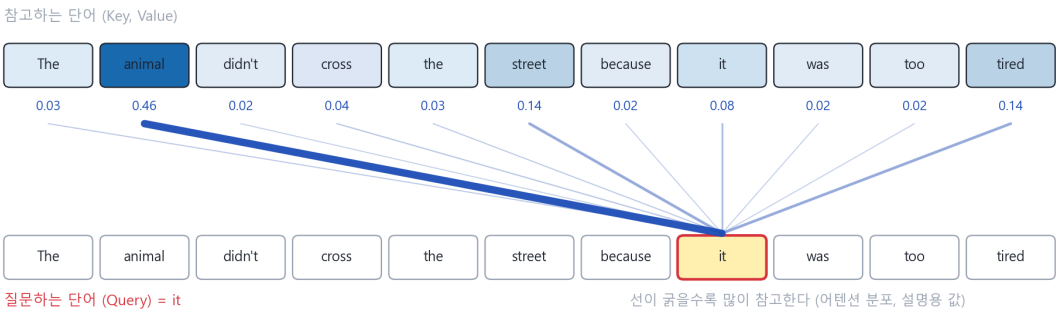



그림 5 it이 문장의 각 단어를 참고하는 비율(설명용 값). 학습된 모델에서는 animal의 비율이 가장 높게 나온다

## 2) 스케일드 닷-프로덕트 어텐션

정의

**(1) Q, K, V 벡터 만들기.** 단어 벡터($d_{model}$칸)에 가중치 행렬 세 개를 각각 곱해 더 작은 벡터 세 개를 만듭니다.

- $W^Q, W^K, W^V$: 모양이 $(d_{model}, d_k)$인 가중치 행렬. **학습으로 정해집니다.** 어떤 단어를 참고할지를 모델이 배우는 곳이 바로 여기입니다.
- $d_k$: Q, K, V 벡터의 크기. $d_k = d_{model} \div \text{num_heads}$입니다. 논문에서는 $512 \div 8 = 64$입니다(왜 나누는지는 ④부).
- 한 단어가 벡터 세 개를 갖습니다. 질문할 때 쓰는 $q$, 다른 단어의 질문과 맞춰 볼 때 쓰는 $k$, 내용을 내줄 때 쓰는 $v$ 입니다.

**(2) 점수 내기.** 쿼리 벡터와 키 벡터의 내적을 $\sqrt{d_k}$로 나눕니다.

$$\text{score}(q, k) = \frac{q \cdot k}{\sqrt{d_k}}$$

내적(dot product)은 두 벡터의 같은 칸끼리 곱해 모두 더한 값이고, 두 벡터가 같은 방향일수록 큽니다(11-04). 내적을 쓰고 $\sqrt{d_k}$로 크기를 줄인다고 해서 스케일드 닷-프로덕트 어텐션(scaled dot-product attention)이라고 부릅니다.

**왜 $\sqrt{d_k}$로 나눌까?** 벡터가 길수록(칸이 많을수록) 내적은 더 많은 항을 더하므로 값이 커집니다. 점수가 크면 소프트맥스가 한 단어에 거의 1을, 나머지에 거의 0을 주게 됩니다. 예를 들어 점수가 [8, 4]이면 비율은 [0.98, 0.02]이지만, 8로 나눈 [1, 0.5]이면 [0.62, 0.38]입니다. 비율이 한쪽으로 쏠리면 다른 단어의 정보를 섞지 못하고, 소프트맥스의 기울기도 0에 가까워져 학습이 느려집니다.

**(3) 비율로 바꾸고 섞기.** 점수에 소프트맥스를 씌워 합이 1인 어텐션 분포를 얻고, 그 비율로 V 벡터들을 가중합합니다. 결과가 그 단어의 어텐션 값입니다.

**(4) 행렬로 한꺼번에.** 단어마다 따로 하지 않고 문장 행렬 전체에 한 번에 계산합니다.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$Q, K, V$는 단어마다의 벡터를 행으로 쌓은 행렬입니다. $K^T$는 $K$의 행과 열을 바꾼 것(전치)입니다. $QK^T$의 (i행, j열) 칸은 i번째 단어의 $q$와 j번째 단어의 $k$의 내적입니다. 소프트맥스는 행마다 따로 씌웁니다.

| 단계 | 계산 | 모양 |
|---|---|---|
| 문장 행렬 | $X$ | (seq_len, $d_{model}$) |
| Q, K, V 행렬 | $XW^Q$, $XW^K$, $XW^V$ | (seq_len, $d_k$) |
| 스코어 행렬 | $QK^T \div \sqrt{d_k}$ | (seq_len, seq_len) |
| 어텐션 분포 | 행마다 소프트맥스 | (seq_len, seq_len), 행마다 합 1 |
| 어텐션 값 행렬 | 분포 × $V$ | (seq_len, $d_k$) |

예시 적용

"I am a student", $d_{model} = 4$, $d_k = 2$로 계산합니다(실습 2). 문장 행렬과 가중치는 학습되지 않은 무작위 값이라, 숫자에 뜻은 없고 계산의 흐름만 봅니다. 단어 I를 쿼리로 두고 따라갑니다.

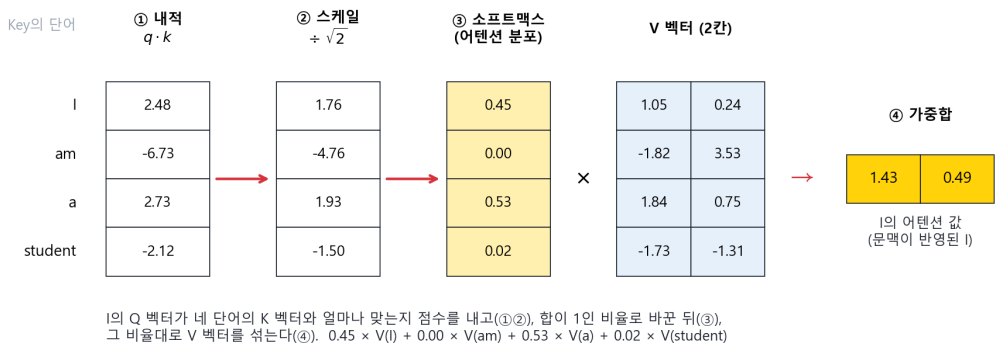



그림 6 I의 Q 벡터 하나로 스케일드 닷-프로덕트 어텐션을 따라간다(실습 2의 실제 값)

- ① I의 $q$와 네 단어의 $k$의 내적: 2.48, -6.73, 2.73, -2.12
- ② $\sqrt{2} = 1.41$로 나눈 스코어: 1.76, -4.76, 1.93, -1.50
- ③ 소프트맥스: 0.45, 0.00, 0.53, 0.02 (합 1). I는 자기 자신을 45%, a를 53% 참고하고 am은 거의 참고하지 않습니다.
- ④ 그 비율로 V 벡터를 섞은 어텐션 값: [1.43, 0.49]. 첫 칸은 $0.45 \times 1.05 + 0.00 \times (-1.82) + 0.53 \times 1.84 + 0.02 \times (-1.73) = 1.43$입니다.

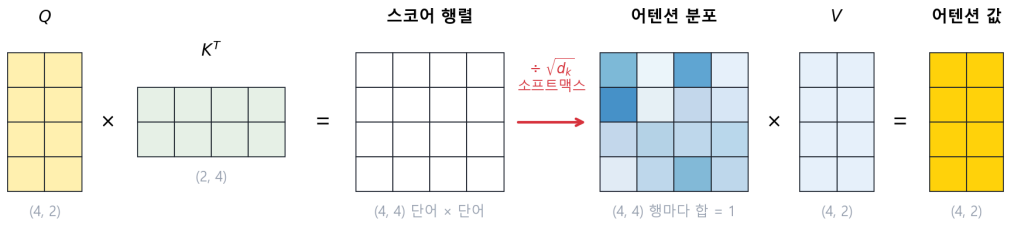



그림 7 같은 계산을 행렬로 한꺼번에 한다. 가운데 어텐션 분포의 색은 실습 2의 실제 값이다(진할수록 큰 비율)

행렬로 계산한 결과의 첫 행은 위에서 따로 구한 I의 값과 같고(True), 전체 결과는 파이토치의 `F.scaled_dot_product_attention`과 같습니다(True).

비유

### 도서관에서 자료 찾기

궁금한 것(질문)을 들고 도서관에 갑니다. 책등의 제목과 내 질문을 맞춰 보고 관련 있어 보이는 책에 점수를 매깁니다. 한 권만 읽지 않고, 점수가 높은 책은 많이, 낮은 책은 조금 읽어 내용을 섞어 정리합니다. 셀프 어텐션에서는 문장의 단어 하나하나가 질문자이면서 동시에 책입니다.

| | |
|---|---|
| 내 질문 | = = 쿼리 Q |
| 책등의 제목 | = = 키 K |
| 책의 내용 | = = 값 V |
| 책마다 읽는 비중 | = = 어텐션 분포 |
| 섞어서 정리한 노트 | = = 어텐션 값 |

실습 2 스케일드 닷-프로덕트 어텐션을 식 그대로 계산하기

```python
import math

import torch
import torch.nn.functional as F

torch.manual_seed(0)
words = ["I", "am", "a", "student"]
d_model, d_k = 4, 2                        # 논문은 512와 64. 여기서는 작게

X = torch.randn(4, d_model)                # 문장 행렬 (단어 4개, d_model). 설명용 값
W_Q = torch.randn(d_model, d_k)            # 학습되는 가중치 행렬 세 개 (d_model, d_k)
W_K = torch.randn(d_model, d_k)
W_V = torch.randn(d_model, d_k)

# 1) 문장 행렬에 가중치 행렬을 곱해 Q, K, V 행렬을 만든다
Q, K, V = X @ W_Q, X @ W_K, X @ W_V
print("X:", tuple(X.shape), "→ Q, K, V:", tuple(Q.shape), tuple(K.shape),
      tuple(V.shape))

# 2) 단어 'I' 하나만 따라가기
q = Q[0]                                                # I의 Q 벡터 (2,)
dots = torch.stack([q @ k for k in K])                  # 모든 K 벡터와 내적
scores = dots / math.sqrt(d_k)                          # √d_k 로 나눔
weights = torch.softmax(scores, dim=0)                  # 어텐션 분포 (합 = 1)
value = sum(w * v for w, v in zip(weights, V))          # V 벡터들의 가중합
print("\n[I의 Q 벡터로]")
print("  내적          :", [f"{v:.2f}" for v in dots.tolist()])
print("  ÷ √2 (스코어) :", [f"{v:.2f}" for v in scores.tolist()])
print("  소프트맥스    :", [f"{v:.2f}" for v in weights.tolist()],
      " 합:", round(weights.sum().item(), 2))
print("  어텐션 값     :", [f"{v:.2f}" for v in value.tolist()])

# 3) 행렬 연산으로 네 단어를 한꺼번에: softmax(Q Kᵀ / √d_k) V
score_matrix = Q @ K.T / math.sqrt(d_k)                 # (4, 4)
attn = torch.softmax(score_matrix, dim=-1)              # 행마다 합이 1
out = attn @ V                                          # (4, 2)
print("\n스코어 행렬:", tuple(score_matrix.shape),
      "→ 어텐션 값 행렬:", tuple(out.shape))
print("어텐션 분포 (행 = 질문하는 단어, 열 = 참고하는 단어)")
print("          " + "".join(f"{w:>9}" for w in words))
for w, row in zip(words, attn):
    print(f"{w:>9}" + "".join(f"{v:9.2f}" for v in row.tolist()))
print("행렬의 첫 행 = 위에서 따로 구한 I의 값?", torch.allclose(out[0], value))

# 4) 파이토치의 함수와 비교
same = torch.allclose(out, F.scaled_dot_product_attention(Q, K, V), atol=1e-6)
print("직접 계산 = F.scaled_dot_product_attention?", same)
torch.save({"dots": dots, "scores": scores, "weights": weights, "value": value,
            "V": V, "attn": attn}, "self_attention_trace.pt")
```

실행 결과

```
X: (4, 4) → Q, K, V: (4, 2) (4, 2) (4, 2)

[I의 Q 벡터로]
  내적          : ['2.48', '-6.73', '2.73', '-2.12']
  ÷ √2 (스코어) : ['1.76', '-4.76', '1.93', '-1.50']
  소프트맥스    : ['0.45', '0.00', '0.53', '0.02']  합: 1.0
  어텐션 값     : ['1.43', '0.49']

스코어 행렬: (4, 4) → 어텐션 값 행렬: (4, 2)
어텐션 분포 (행 = 질문하는 단어, 열 = 참고하는 단어)
                I       am        a  student
        I    0.45     0.00     0.53     0.02
       am    0.62     0.04     0.22     0.11
        a    0.23     0.27     0.25     0.26
  student    0.07     0.25     0.43     0.25
행렬의 첫 행 = 위에서 따로 구한 I의 값? True
직접 계산 = F.scaled_dot_product_attention? True
```

직접 해 보기 `scores = dots / math.sqrt(d_k)`에서 나누기를 빼고 소프트맥스를 구해 보세요. I의 분포가 어떻게 달라지나요? 반대로 100으로 나누면 어떻게 되나요?

## ③ 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

**1** 셀프 어텐션에서 Q, K, V는 모두 같은 문장의 단어 벡터에서 만들어진다. (O / X) 이름 _____

**2** 어텐션 분포는 한 행의 값을 모두 더하면 1이 된다. (O / X) 이름 _____

**3** 어텐션 스코어를 구할 때 쓰는 계산은? 이름 _____

① Q와 K의 내적을 $\sqrt{d_k}$로 나눈다 ② Q와 V를 더한다 ③ K와 V의 내적

**4** 단어가 4개인 문장의 스코어 행렬의 모양은? 이름 _____

① (4, 2) ② (4, 4) ③ (2, 2)

**5** 논문에서 $d_k = d_{model} \div \text{num_heads} = 512 \div 8 =$ _____이다. 이름 _____

**6** 어텐션 값은 어텐션 분포의 비율로 _____ 벡터들을 가중합한 것이다. 이름 _____

**7** 셀프 어텐션의 네 단계(점수, 스케일, 소프트맥스, 가중합)를 도서관에서 자료 찾기에 빗대어 설명해 보세요. 이름 _____

# 18-01 ④ 멀티 헤드 어텐션과 패딩 마스크

핵심 한 줄 **멀티 헤드 어텐션은 작은 어텐션 여러 개를 서로 다른 가중치로 동시에 하고 결과를 이어 붙이는 방법이고, 패딩 마스크는 뜻 없는 <PAD>가 참고 대상이 되지 않도록 가리는 장치다.**

왜 배우나 단어 사이의 관계는 한 가지가 아닙니다. it이 무엇을 가리키는지, 주어의 동사가 무엇인지, 꾸미는 말이 무엇인지가 모두 다른 관계입니다. 어텐션 한 번으로는 비율이 하나뿐이라 한 가지 관계만 담을 수 있습니다. ③부에서 벡터를 $d_{model}$보다 작은 $d_k$로 줄인 이유가 여기서 나옵니다.

## 1) 멀티 헤드 어텐션

정의

- 멀티 헤드 어텐션(multi-head attention): ③부의 어텐션을 `num_heads` 개 동시에 하는 것. 하나하나를 어텐션 헤드(head)라고 합니다.
- 헤드마다 $W^Q, W^K, W^V$가 **따로** 있습니다. 그래서 헤드마다 다른 단어에 주목하도록 학습될 수 있습니다.
- 헤드 하나의 크기는 $d_k = d_{model} \div \text{num_heads}$입니다. 큰 어텐션 하나를 하는 대신 작은 어텐션 여러 개를 하므로, 전체 계산량은 비슷합니다.

$$\underbrace{X}_{(L,\, d_{model})} \longrightarrow \underbrace{\text{head}_1, \ldots, \text{head}_h}_{h\times(L,\, d_k)} \xrightarrow{\text{concat}} (L,\ h \times d_k) = (L,\ d_{model}) \xrightarrow{\times W^O} (L,\ d_{model})$$

$L$은 seq_len, $h$는 num_heads입니다. 헤드마다 나온 어텐션 값 행렬 $(L, d_k)$을 옆으로 이어 붙이면(concatenate) 다시 $(L, d_{model})$이 됩니다. 여기에 가중치 행렬 $W^O$ $(d_{model}, d_{model})$를 한 번 더 곱합니다. $W^O$는 여러 헤드가 따로 모은 정보를 하나로 섞는 역할을 하고, 역시 학습됩니다.

**기억할 것.** 멀티 헤드 어텐션에 들어간 모양과 나온 모양이 같습니다. (seq_len, $d_{model}$)이 들어가서 (seq_len, $d_{model}$)이 나옵니다.

예시 적용 "I am a student", $d_{model} = 4$, 헤드 2개($d_k = 2$)로 계산합니다(실습 3).

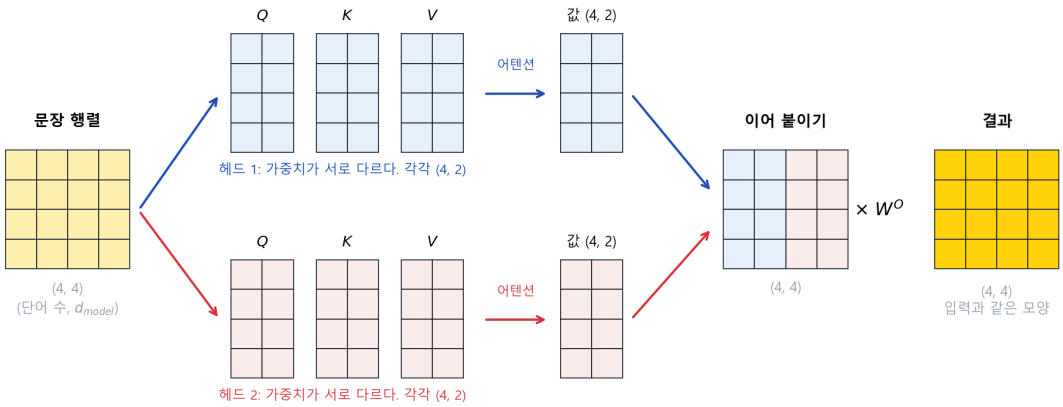



그림 8 헤드마다 따로 Q, K, V를 만들어 어텐션을 하고, 결과 (4, 2) 두 개를 이어 붙여 (4, 4)로 되돌린다

- 헤드 1에서 I의 분포는 [0.38, 0.37, 0.21, 0.03], 헤드 2에서는 [0.24, 0.24, 0.28, 0.24]입니다. 같은 문장인데 헤드마다 보는 곳이 다릅니다(학습되지 않은 값이라 뜻은 없습니다).
- 파이토치의 `nn.MultiheadAttention`에서 가중치를 꺼내 헤드별로 나눠 직접 계산한 결과가 라이브러리의 결과와 같습니다(True).
- 입력 (1, 4, 4)가 출력 (1, 4, 4)로, 모양이 그대로입니다. 맨 앞의 1은 문장의 수(배치)입니다.

학습된 모델이라면, it을 쿼리로 했을 때 한 헤드는 animal(가리키는 대상)에, 다른 헤드는 tired(상태)에 큰 비율을 주는 식으로 역할이 나뉩니다.

## 2) 패딩 마스크

정의

- 마스킹(masking): 어텐션에서 특정 위치를 참고하지 못하게 가리는 것.
- 패딩 마스크(padding mask): 길이를 맞추려고 넣은 <PAD>를 가리는 마스크. <PAD>는 뜻이 없으므로 다른 단어가 참고해서는 안 됩니다.
- 스코어 행렬에서 행은 Query, 열은 Key입니다. **Key가 <PAD>인 열 전체**를 가립니다.

**가리는 방법.** 소프트맥스를 씌우기 **전**의 스코어에서 가릴 자리에 아주 작은 값(-1,000,000,000 같은, 음의 무한대에 가까운 수)을 넣습니다. 소프트맥스는 $e^{\text{score}}$를 쓰므로 $e^{-10^9} \approx 0$이 되어 그 자리의 비율이 0이 되고, 나머지 자리끼리 합이 1이 되도록 나눠 갖습니다.

**왜 소프트맥스 뒤에 0을 넣지 않을까?** 뒤에서 0으로 바꾸면 남은 비율의 합이 1보다 작아집니다. 앞에서 가려야 남은 단어들끼리 합이 1인 분포가 됩니다.

13-03에서 RNN은 뒤에 붙은 패딩을 읽느라 내용이 흐려졌습니다. 트랜스포머는 패딩을 아예 참고하지 않으므로 그런 문제가 없습니다.

예시 적용

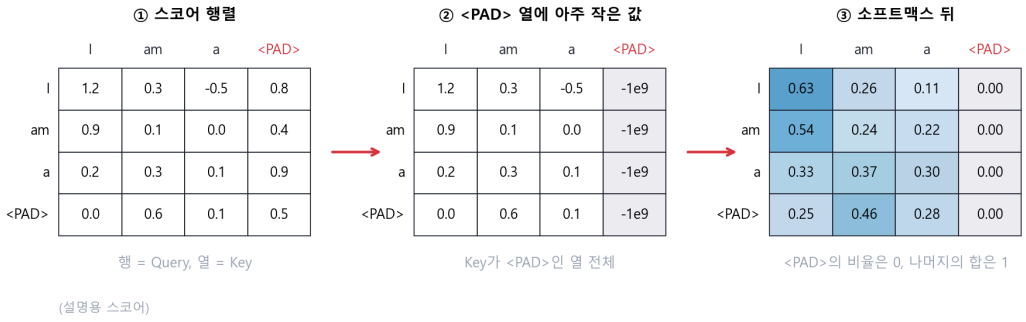



그림 9 "I am a <PAD>"의 스코어 행렬에서 <PAD> 열을 가리면, 소프트맥스 뒤에 그 열의 비율이 모두 0이 된다

첫 행의 스코어 [1.2, 0.3, -0.5, 0.8]에서 마지막 칸을 -1e9로 바꾸고 소프트맥스를 씌우면 [0.63, 0.26, 0.11, 0.00]입니다(실습 3). 파이토치에서는 `key_padding_mask`에 가릴 자리를 `True`로 넘깁니다. 넘긴 뒤의 어텐션 분포는 네 행 모두 마지막 칸이 0.00이고 합은 1입니다.

비유

### 여러 색 형광펜으로 읽기

같은 글을 여러 번 읽으며, 노란 펜으로는 "누가"를, 초록 펜으로는 "무엇을"을, 분홍 펜으로는 "가리키는 말이 무엇인지"를 표시합니다. 색마다 눈여겨보는 것이 다릅니다. 마지막에 세 가지 표시를 한 장에 모아 정리합니다. 글 끝의 빈 줄에는 어떤 펜으로도 표시하지 않습니다.

| | |
|---|---|
| 형광펜 한 색 | = = 어텐션 헤드 하나 |
| 색마다 다른 기준 | = = 헤드마다 다른 $W^Q, W^K, W^V$ |
| 표시를 한 장에 모으기 | = = 이어 붙이기 + $W^O$ |
| 빈 줄에는 표시하지 않기 | = = 패딩 마스크 |

실습 3 멀티 헤드 어텐션을 직접 계산하고, 패딩 마스크 걸기

```python
import math

import torch
import torch.nn as nn

torch.manual_seed(0)
d_model, num_heads = 4, 2
d_k = d_model // num_heads                # 헤드 하나의 크기 = 4 / 2 = 2
mha = nn.MultiheadAttention(d_model, num_heads, batch_first=True)
X = torch.randn(1, 4, d_model)            # (문장 1개, 단어 4개, d_model)

# 1) 파이토치의 멀티 헤드 어텐션: Q, K, V 자리에 모두 X를 넣으면 셀프 어텐션
out, attn = mha(X, X, X, average_attn_weights=False)
print("입력:", tuple(X.shape), "→ 출력:", tuple(out.shape), "(모양이 그대로)")
print("어텐션 분포:", tuple(attn.shape), "= (문장, 헤드 수, 단어, 단어)")

# 2) 같은 계산을 직접: 파이토치는 W_Q, W_K, W_V를 한 행렬에 이어 붙여 둔다
W_Q, W_K, W_V = mha.in_proj_weight.chunk(3)          # 각각 (4, 4)
b_Q, b_K, b_V = mha.in_proj_bias.chunk(3)
x = X[0]                                             # (4, 4)
Q, K, V = x @ W_Q.T + b_Q, x @ W_K.T + b_K, x @ W_V.T + b_V
heads = []
for h in range(num_heads):
    cols = slice(h * d_k, (h + 1) * d_k)             # 헤드마다 2칸씩 나눠 쓴다
    q, k, v = Q[:, cols], K[:, cols], V[:, cols]     # 각각 (4, 2)
    a = torch.softmax(q @ k.T / math.sqrt(d_k), dim=-1)
    heads.append(a @ v)                              # 헤드의 어텐션 값 (4, 2)
    print(f"헤드 {h + 1}: Q, K, V {tuple(q.shape)} → 값 {tuple(heads[-1].shape)},",
          "'I'의 분포", [f"{t:.2f}" for t in a[0].tolist()])
concat = torch.cat(heads, dim=-1)                    # 이어 붙이기 (4, 4)
manual = concat @ mha.out_proj.weight.T + mha.out_proj.bias   # W_O 곱하기
print("이어 붙이기:", tuple(concat.shape), "→ W_O 곱한 결과:", tuple(manual.shape))
print("직접 계산 = nn.MultiheadAttention?",
      torch.allclose(manual, out[0], atol=1e-6))

# 3) 패딩 마스크: 마지막 단어가 <PAD>라면 그 열을 보지 않게 한다
pad_mask = torch.tensor([[False, False, False, True]])   # True = 가릴 자리
_, attn_pad = mha(X, X, X, key_padding_mask=pad_mask)
print("\n패딩 마스크를 준 뒤의 어텐션 분포 (헤드 평균)")
for row in attn_pad[0]:
    print("  ", [f"{t:.2f}" for t in row.tolist()], " 합:",
          round(row.sum().item(), 2))

# 4) 마스크가 하는 일을 직접: 스코어에 아주 작은 값을 넣고 소프트맥스
scores = torch.tensor([1.2, 0.3, -0.5, 0.8])
masked = scores.masked_fill(pad_mask[0], -1e9)
print("스코어:", [1.2, 0.3, -0.5, 0.8], "→ 마지막 칸을 -1e9로 바꾼다")
print("소프트맥스:", [f"{t:.2f}" for t in torch.softmax(masked, dim=0).tolist()])
```

실행 결과

```
입력: (1, 4, 4) → 출력: (1, 4, 4) (모양이 그대로)
어텐션 분포: (1, 2, 4, 4) = (문장, 헤드 수, 단어, 단어)
헤드 1: Q, K, V (4, 2) → 값 (4, 2), 'I'의 분포 ['0.38', '0.37', '0.21', '0.03']
헤드 2: Q, K, V (4, 2) → 값 (4, 2), 'I'의 분포 ['0.24', '0.24', '0.28', '0.24']
이어 붙이기: (4, 4) → W_O 곱한 결과: (4, 4)
직접 계산 = nn.MultiheadAttention? True

패딩 마스크를 준 뒤의 어텐션 분포 (헤드 평균)
   ['0.36', '0.35', '0.29', '0.00']  합: 1.0
   ['0.55', '0.23', '0.22', '0.00']  합: 1.0
   ['0.34', '0.35', '0.31', '0.00']  합: 1.0
   ['0.25', '0.47', '0.28', '0.00']  합: 1.0
스코어: [1.2, 0.3, -0.5, 0.8] → 마지막 칸을 -1e9로 바꾼다
소프트맥스: ['0.63', '0.26', '0.11', '0.00']
```

직접 해 보기 `num_heads`를 4로 바꾸면 헤드 하나의 크기 `d_k`는 얼마가 되나요? 3으로 바꾸면 왜 오류가 날까요?

## ④ 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

**1** 멀티 헤드 어텐션의 헤드들은 모두 같은 가중치 행렬을 쓴다. (O / X) 이름 _____

**2** 멀티 헤드 어텐션을 지나면 행렬의 모양이 (seq_len, $d_{model}$)에서 달라진다. (O / X) 이름 _____

**3** 헤드마다 나온 어텐션 값 행렬을 합치는 방법은? 이름 _____

① 옆으로 이어 붙인다 ② 평균을 낸다 ③ 가장 큰 것만 고른다

**4** 패딩 마스크가 가리는 것은? 이름 _____

① Key가 <PAD>인 열 ② 문장의 첫 단어 ③ 점수가 가장 큰 칸

**5** 마스킹할 자리에는 소프트맥스 _____(전 / 후)에 아주 작은 음수를 넣는다. 이름 _____

**6** $d_{model} = 512$, `num_heads` = 8이면 헤드 하나의 크기는 _____이다. 이름 _____

**7** 어텐션을 한 번 크게 하지 않고 여러 헤드로 나누는 이유를 여러 색 형광펜에 빗대어 설명해 보세요. 이름 _____

# 18-01 ⑤ 인코더 층 완성: FFNN, 잔차 연결, 층 정규화

핵심 한 줄 **인코더 층은 멀티 헤드 셀프 어텐션과 피드 포워드 신경망 두 서브층으로 이루어지고, 서브층마다 입력을 출력에 더한 뒤(잔차 연결) 값의 크기를 맞춰(층 정규화) 여러 층을 쌓아도 학습이 되게 한다.**

왜 배우나 어텐션만으로는 층이 완성되지 않습니다. 섞은 정보를 가공하는 부품(FFNN)과, 층을 6개, 12개 쌓아도 학습이 무너지지 않게 하는 안전장치(Add & Norm)가 필요합니다. 이 세 가지를 붙이면 인코더 층 하나가 완성되고, 디코더도 같은 부품을 그대로 씁니다.

## 1) 포지션-와이즈 피드 포워드 신경망

정의

$$\text{FFNN}(x) = \max(0,\ xW_1 + b_1)\,W_2 + b_2$$

- 06장에서 본 완전 연결 신경망(은닉층 1개)입니다. $\max(0, \cdot)$은 ReLU 활성화 함수입니다.
- $x$: 멀티 헤드 어텐션을 지난 행렬 (seq_len, $d_{model}$)
- $W_1$: ($d_{model}$, $d_{ff}$), $W_2$: ($d_{ff}$, $d_{model}$). 안에서 $d_{ff}$로 넓어졌다가 다시 $d_{model}$로 돌아옵니다. 논문에서는 512 → 2048 → 512입니다.
- 포지션-와이즈(position-wise): 단어의 자리마다 **같은 가중치**를 **따로** 적용한다는 뜻입니다. 이 서브층에서는 단어끼리 정보를 주고받지 않습니다.
- $W_1, b_1, W_2, b_2$는 한 층 안에서는 모든 단어가 함께 쓰고, 층마다는 다른 값을 가집니다.

**왜 필요할까?** 어텐션이 하는 일은 V 벡터들의 가중합, 곧 "섞기"입니다. 섞은 결과를 비선형 함수(ReLU)로 가공하는 단계가 있어야 복잡한 관계를 표현할 수 있습니다(06-03에서 층과 활성화 함수가 있어야 XOR을 풀 수 있었던 것과 같습니다). 어텐션이 "다른 단어에서 정보를 모으는 일"이라면 FFNN은 "모은 정보를 각자 소화하는 일"입니다.

예시 적용 $d_{model} = 4$, $d_{ff} = 16$인 FFNN에 단어 4개의 행렬 (4, 4)를 넣으면 (4, 4)가 나옵니다(실습 4). 행렬로 한꺼번에 넣은 결과와 단어 하나씩 따로 넣어 모은 결과가 같습니다(True). "포지션-와이즈"가 뜻하는 바입니다.

## 2) 잔차 연결과 층 정규화 (Add & Norm)

정의

$$\text{output} = \text{LayerNorm}\big(x + \text{Sublayer}(x)\big)$$

$x$는 서브층의 입력, $\text{Sublayer}(x)$는 서브층(어텐션 또는 FFNN)의 출력입니다. 두 서브층 모두 뒤에 이 계산이 붙습니다. 그림에서는 **Add & Norm**이라고 적습니다.

**(1) 잔차 연결(residual connection)**: 서브층의 입력을 출력에 **더하는** 것. $H(x) = x + F(x)$.

- 더하려면 모양이 같아야 합니다. 모든 서브층이 (seq_len, $d_{model}$)을 유지하도록 만든 이유입니다.
- **왜 하는가 ①**: 서브층은 처음부터 새 값을 만들 필요 없이 "입력에서 무엇을 고칠지($F(x)$)"만 배우면 됩니다. 원래 정보(단어의 뜻, 위치)는 더하기를 통해 그대로 위층으로 올라갑니다.
- **왜 하는가 ②**: 역전파에서 기울기가 지나갈 지름길이 생깁니다. $\partial H/\partial x = 1 + \partial F/\partial x$라서, 서브층의 기울기가 작아져도 1이 남습니다. 층을 깊게 쌓을 때의 기울기 소실(06-08)을 막습니다.

**(2) 층 정규화(layer normalization)**: 단어 벡터 하나하나를, 그 벡터 안의 값들이 평균 0, 분산 1이 되도록 바꾸는 것.

$$\hat{x}_{i,k} = \frac{x_{i,k} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}, \qquad ln_i = \gamma\,\hat{x}_i + \beta$$

- $x_i$: i번째 단어의 벡터. $x_{i,k}$는 그 벡터의 k번째 칸
- $\mu_i$, $\sigma_i^2$: 그 벡터의 $d_{model}$개 값의 평균과 분산. 단어마다 따로 구합니다.
- $\epsilon$(입실론): 분모가 0이 되지 않게 더하는 아주 작은 수

- $\gamma$(감마), $\beta$(베타): 크기가 $d_{model}$인 학습되는 벡터. 1과 0에서 시작합니다. 평균 0, 분산 1로 묶어 두기만 하면 표현이 제한되므로, 필요한 만큼 크기와 위치를 다시 조절할 수 있게 둔 것입니다.

**왜 하는가.** 잔차 연결로 값을 계속 더하면 층을 지날수록 값이 커지거나 들쭉날쭉해질 수 있습니다. 층마다 크기를 일정하게 맞춰 주면 다음 층이 늘 비슷한 범위의 값을 받아 학습이 안정됩니다. 06-08의 배치 정규화와 목적은 같고, 평균을 구하는 범위가 "배치 방향"이 아니라 "한 단어의 벡터 안"이라는 점이 다릅니다. 문장의 길이나 배치의 크기와 상관없이 쓸 수 있습니다.

### 예시 적용

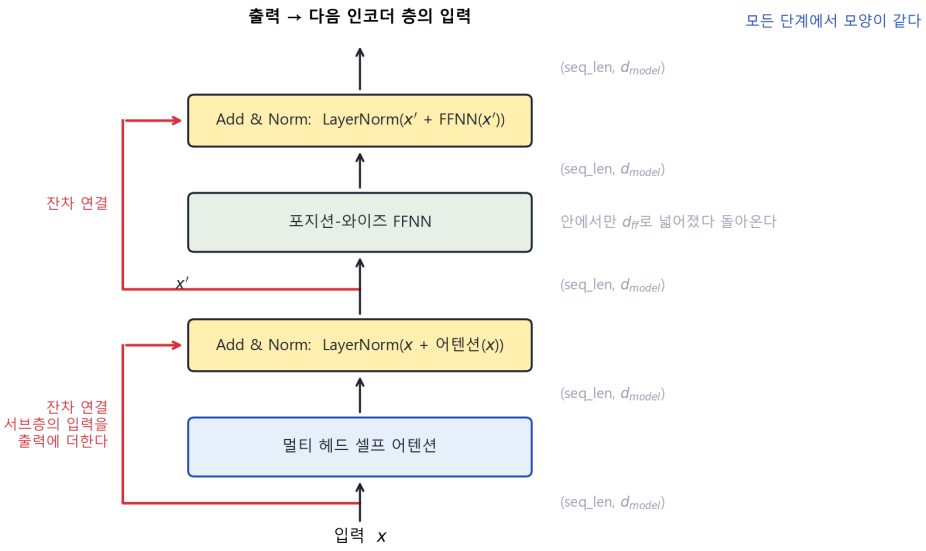



그림 10 인코더 층 하나. 빨간 선이 잔차 연결이다. 어느 단계에서 재어도 모양은 (seq_len, $d_{model}$)이다

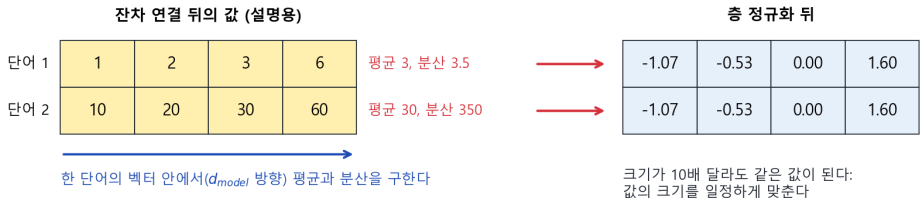



그림 11 단어 벡터 [1, 2, 3, 6]과 그 10배인 [10, 20, 30, 60]은 층 정규화 뒤에 같은 값이 된다(실습 4)

- [1, 2, 3, 6]의 평균은 3, 분산은 3.5입니다. $(1-3) \div \sqrt{3.5} = -1.07$처럼 계산하면 [-1.07, -0.53, 0.00, 1.60]이 됩니다. 직접 계산한 값이 `nn.LayerNorm`과 같습니다(True).
- **논문 크기의 인코더 층**: (1, 4, 512)를 넣으면 층 1개를 지나도, 6개를 지나도 (1, 4, 512)입니다.
- **파라미터 수**: 층 하나에 3,152,384개입니다. 어텐션이 1,050,624개, FFNN이 2,099,712개, 층 정규화 2개가 2,048개입니다. FFNN이 어텐션의 두 배입니다. 6층이면 18,914,304개입니다.

비유

### 원고에 덧붙이는 교정 메모

여러 사람이 차례로 원고를 고칩니다. 각자 원고를 처음부터 새로 쓰면, 몇 사람만 거쳐도 원래 내용이 사라집니다. 그래서 원문은 그대로 두고 고칠 내용만 메모로 덧붙여 다음 사람에게 넘깁니다. 메모가 쌓여 지저분해지지 않도록, 넘기기 전에 한 번씩 깨끗하게 정서합니다.

| | |
|---|---|
| 원문 | = = 서브층의 입력 $x$ |
| 교정 메모 | = = 서브층의 출력 $F(x)$ |
| 원문에 메모 덧붙이기 | = = 잔차 연결 $x + F(x)$ |
| 넘기기 전 정서 | = = 층 정규화 |

실습 4

## 층 정규화, FFNN, 그리고 논문 크기의 인코더 층

```python
import torch
import torch.nn as nn

torch.manual_seed(0)

# 1) 층 정규화를 식 그대로: 단어 벡터 하나(행)마다 평균 0, 분산 1로 맞춘다
x = torch.tensor([[1.0, 2.0, 3.0, 6.0],
                  [10.0, 20.0, 30.0, 60.0]])         # 단어 2개, d_model = 4
mean = x.mean(dim=-1, keepdim=True)                  # 행마다 평균
var = x.var(dim=-1, keepdim=True, unbiased=False)    # 행마다 분산
x_hat = (x - mean) / torch.sqrt(var + 1e-5)
layer_norm = nn.LayerNorm(4)                         # γ = 1, β = 0 에서 시작
print("평균:", mean.flatten().tolist(), " 분산:", var.flatten().tolist())
for row in x_hat:
    print("  정규화 후:", [f"{v:.2f}" for v in row.tolist()])
print("직접 계산 = nn.LayerNorm?", torch.allclose(x_hat, layer_norm(x), atol=1e-5))

# 2) 포지션-와이즈 FFNN: d_model → d_ff → d_model
d_model, d_ff = 4, 16
ffnn = nn.Sequential(nn.Linear(d_model, d_ff), nn.ReLU(), nn.Linear(d_ff, d_model))
X = torch.randn(4, d_model)                          # 단어 4개
together = ffnn(X)                                   # 행렬로 한꺼번에
one_by_one = torch.stack([ffnn(row) for row in X])   # 단어 하나씩 따로
print("\nFFNN:", tuple(X.shape), "→", tuple(together.shape))
print("한꺼번에 = 단어마다 따로?", torch.allclose(together, one_by_one, atol=1e-6))

# 3) 잔차 연결 + 층 정규화 (Add & Norm): LayerNorm(x + Sublayer(x))
add_norm = layer_norm(X + ffnn(X))
print("Add & Norm 뒤의 모양:", tuple(add_norm.shape))

# 4) 논문 크기의 인코더 층 하나, 그리고 6층
layer = nn.TransformerEncoderLayer(d_model=512, nhead=8, dim_feedforward=2048,
                                   batch_first=True)
encoder = nn.TransformerEncoder(layer, num_layers=6, enable_nested_tensor=False)
sentence = torch.randn(1, 4, 512)                    # (문장 1개, 단어 4개, 512)
print("\n인코더 층 1개:", tuple(sentence.shape), "→", tuple(layer(sentence).shape))
print("인코더 6층   :", tuple(sentence.shape), "→", tuple(encoder(sentence).shape))

def count(module):
    return sum(p.numel() for p in module.parameters())

attn_params = count(layer.self_attn)
ffnn_params = count(layer.linear1) + count(layer.linear2)
print(f"층 하나의 파라미터: {count(layer):,}개",
      f"(어텐션 {attn_params:,} + FFNN {ffnn_params:,} + 층 정규화 2개)")
print(f"6층의 파라미터: {count(encoder):,}개")
```

실행 결과

```
평균: [3.0, 30.0]  분산: [3.5, 350.0]
  정규화 후: ['-1.07', '-0.53', '0.00', '1.60']
  정규화 후: ['-1.07', '-0.53', '0.00', '1.60']
직접 계산 = nn.LayerNorm? True

FFNN: (4, 4) → (4, 4)
한꺼번에 = 단어마다 따로? True
Add & Norm 뒤의 모양: (4, 4)
```

```
인코더 층 1개: (1, 4, 512) → (1, 4, 512)
인코더 6층   : (1, 4, 512) → (1, 4, 512)
층 하나의 파라미터: 3,152,384개 (어텐션 1,050,624 + FFNN 2,099,712 + 층 정규화 2개)
6층의 파라미터: 18,914,304개
```

직접 해 보기 `dim_feedforward`를 2048에서 512로 줄이면 층 하나의 파라미터가 몇 개가 되나요? 어텐션과 FFNN 중 어느 쪽이 줄어드나요?

확인 문제

## ⑤ 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

**1** 포지션-와이즈 FFNN에서는 단어들이 서로 정보를 주고받는다. (O / X) 이름 _____

**2** 잔차 연결은 서브층의 입력과 출력을 더하는 것이다. (O / X) 이름 _____

**3** 층 정규화가 평균과 분산을 구하는 범위는? 이름 _____

① 단어 하나의 벡터 안 ② 배치의 모든 문장 ③ 단어 집합 전체

**4** 잔차 연결을 하려면 서브층의 입력과 출력이 어때야 하나요? 이름 _____

① 모양이 같아야 한다 ② 값이 같아야 한다 ③ 출력이 더 커야 한다

**5** 논문에서 FFNN의 은닉층의 크기 $d_{ff}$는 _____이다. 이름 _____

**6** 그림의 "Add & Norm"에서 Add는 _____ 연결을, Norm은 층 정규화를 뜻한다. 이름 _____

**7** 잔차 연결과 층 정규화를 왜 하는지, 원고의 교정 메모에 빗대어 설명해 보세요. 이름 _____

# 18-01 ⑥ 디코더: 한 단어씩 번역을 만든다

핵심 한 줄

**디코더는 뒤의 단어를 가린 셀프 어텐션(룩-어헤드 마스크)으로 지금까지 만든 번역을 읽고, 인코더-디코더 어텐션으로 입력 문장을 참고해 다음 단어의 확률을 낸다.**

왜 배우나

인코더는 입력 문장을 "이해한 결과"를 내놓을 뿐, 번역을 만들지는 않습니다. 번역을 만드는 쪽이 디코더입니다. 디코더는 인코더의 부품을 그대로 쓰고 두 가지만 다릅니다. 답을 미리 보지 못하게 하는 마스크와, 인코더의 출력을 참고하는 어텐션입니다. 마지막에 모든 부품을 조립한 작은 트랜스포머를 실제로 학습시켜 번역을 확인합니다.

## 1) 마스크드 셀프 어텐션과 룩-어헤드 마스크

정의

- 교사 강요(teacher forcing): 학습할 때 디코더에 **정답 문장**을 입력으로 넣어 주는 방법(16장). 디코더의 입력은 정답 앞에 <sos>를 붙인 것이고, 맞혀야 할 정답은 한 칸 밀린 것입니다.
- 룩-어헤드 마스크(look-ahead mask): 각 단어가 자기보다 **뒤에 있는 단어를 참고하지 못하게** 가리는 마스크. 스코어 행렬에서 대각선의 오른쪽 위 삼각형을 가립니다.
- 마스크드 셀프 어텐션: 룩-어헤드 마스크를 건 셀프 어텐션. 디코더 층의 첫 번째 서브층입니다. 그 밖에는 인코더의 셀프 어텐션과 계산이 같습니다.

**왜 가려야 할까?** RNN 디코더는 단어를 차례로 받으므로 다음 단어를 예측할 때 앞의 단어만 알았습니다. 트랜스포머 디코더는 정답 문장 "<sos> je suis étudiant"를 **한꺼번에** 받습니다. je 자리에서 다음 단어 suis를 맞혀야 하는데, 가리지 않으면 셀프 어텐션으로 바로 옆의 suis를 볼 수 있습니다. 답을 보고 답을 쓰는 셈이라 아무것도 배우지 못합니다. 실제로 번역할 때는 뒤의 단어가 아직 없으므로, 학습 때도 없는 것처럼 만들어야 합니다.

가리는 방법은 패딩 마스크와 같습니다. 소프트맥스 전의 스코어에 아주 작은 값을 넣습니다. 디코더의 입력에 <PAD>가 있으면 그것도 함께 가립니다.

예시 적용

디코더의 입력 "<sos> je suis étudiant"에 룩-어헤드 마스크를 건 어텐션 분포입니다(실습 5, 설명용 스코어).

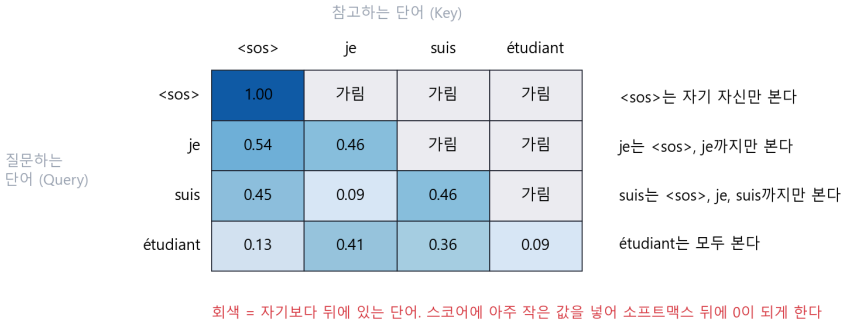



그림 12 행은 질문하는 단어, 열은 참고하는 단어. 각 단어는 자기 자신과 그 앞의 단어만 본다

| 자리 | 디코더의 입력 | 볼 수 있는 단어 | 맞혀야 하는 정답 |
|---|---|---|---|
| 0 | <sos> | <sos> | je |
| 1 | je | <sos>, je | suis |
| 2 | suis | <sos>, je, suis | étudiant |
| 3 | étudiant | <sos>, je, suis, étudiant | <eos> |

**정말 미래를 보지 못하는지 확인.** 마지막 단어 étudiant를 다른 벡터로 바꿔 넣고 앞 세 단어의 출력을 비교했습니다. 마스크가 있으면 앞 세 단어의 출력이 그대로입니다(True). 마스크가 없으면 달라집니다(False). 뒤의 단어가 앞 단어의 계산에 섞여 들어간 것입니다.

## 2) 인코더-디코더 어텐션과 세 가지 어텐션 정리

정의

- 인코더-디코더 어텐션: 디코더 층의 두 번째 서브층. **Query는 디코더**(첫 번째 서브층의 결과)에서, **Key와 Value는 인코더의 마지막 층의 출력**에서 만듭니다.
- Q와 K, V의 출처가 다르므로 셀프 어텐션이 아닙니다. 17장의 어텐션(디코더가 입력 문장을 참고)과 같은 역할입니다.
- 스코어 행렬의 모양은 (출력 문장의 길이, 입력 문장의 길이)입니다. "지금 만들 단어를 위해 입력 문장의 어느 단어를 볼 것인가"를 정합니다.

트랜스포머의 세 가지 어텐션을 한 표로 정리합니다. 셋 다 멀티 헤드 어텐션이고, 안에서 스케일드 닷-프로덕트 어텐션을 씁니다.

| 어텐션 | 있는 곳 | Query | Key, Value | 마스크 |
|---|---|---|---|---|
| 인코더의 셀프 어텐션 | 인코더 층 ① | 입력 문장 | 입력 문장 | 패딩 마스크 |
| 디코더의 마스크드 셀프 어텐션 | 디코더 층 ① | 출력 문장 | 출력 문장 | 룩-어헤드 마스크 (+ 패딩) |
| 인코더-디코더 어텐션 | 디코더 층 ② | 출력 문장 | 입력 문장 (인코더의 출력) | 패딩 마스크 |

**디코더 층의 구성.** 마스크드 셀프 어텐션 → 인코더-디코더 어텐션 → 포지션-와이즈 FFNN이고, 서브층마다 Add & Norm이 붙습니다. 모양은 계속 (출력 문장의 길이, $d_{model}$)입니다. 마지막 디코더 층 뒤에 **선형층**이 $d_{model}$칸을 단어 집합의 크기만큼의 점수로 바꾸고, 소프트맥스가 확률로 바꿉니다(05장).

예시 적용

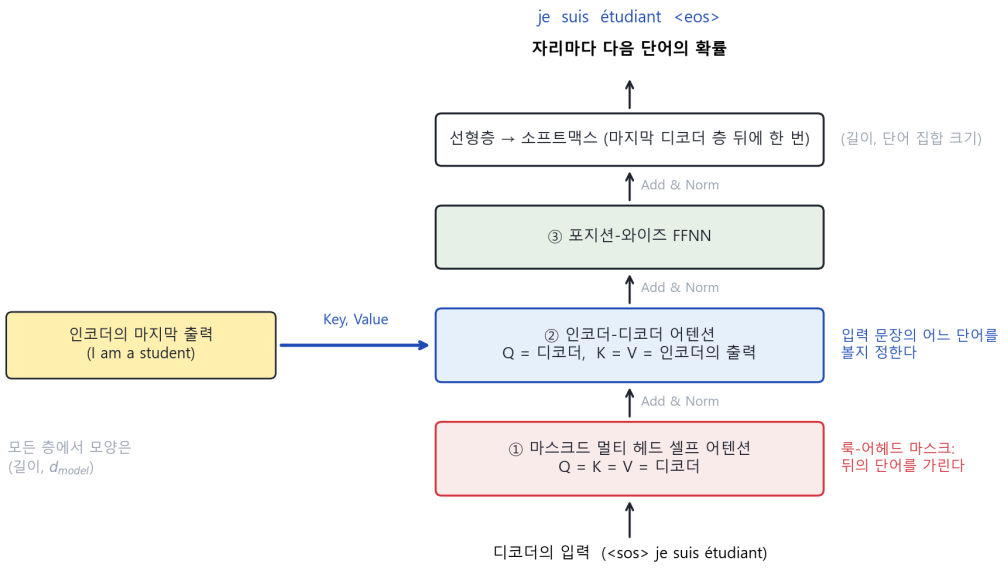



그림 13 디코더 층 하나와 출력. 인코더의 출력은 두 번째 서브층에 Key, Value로 들어간다

### 3) 학습과 번역, 그리고 작은 트랜스포머

정의

| | 학습할 때 | 번역할 때 (추론) |
|---|---|---|
| 디코더의 입력 | 정답 문장 전체 (교사 강요) | <sos>와 지금까지 만든 단어 |
| 한 번에 나오는 것 | 모든 자리의 예측 (병렬) | 마지막 자리의 예측 하나만 씀 |
| 반복 | 문장마다 한 번 | <eos>가 나올 때까지 단어마다 한 번 |
| 룩-어헤드 마스크 | 답을 미리 보지 못하게 하려고 필요 | 학습 때와 같은 조건을 유지하려고 그대로 씀 |

손실은 자리마다의 예측과 정답 사이의 크로스 엔트로피(05-02)이고, <PAD> 자리는 손실에서 뺍니다. 인코더는 번역할 때 처음에 한 번만 계산하고, 디코더만 단어 수만큼 반복합니다.

예시 적용

지금까지의 부품을 조립한 작은 트랜스포머(실습 6)를 작은 번역 데이터로 학습시킵니다(실습 7). 교재에는 이 장의 실습 코드가 없어 이 자료에서 만들었습니다.

| | 논문 | 실습 6의 작은 트랜스포머 |
|---|---|---|
| $d_{model}$ | 512 | 32 |
| `num_layers` | 6 | 2 |
| `num_heads` | 8 | 2 (헤드 하나의 크기 16) |
| $d_{ff}$ | 2048 | 64 |
| 학습 데이터 | 영어-독일어 문장 450만 쌍 | 영어-프랑스어 문장 79개 |

- **데이터**: 주어 5개(I, you, he, we, they) × 서술어 9개(a student, happy, at home, …) × 긍정/부정 = 90문장. 이 중 11문장은 학습에서 빼 두었습니다.
- **모양**: 인코더 입력 (79, 5), 디코더 입력 (79, 8), 모델의 출력 (79, 8, 35). 35는 프랑스어 단어 집합의 크기입니다. 자리마다 35개 단어 각각의 점수가 나옵니다.
- **교사 강요**: 디코더 입력 "<sos> je suis professeur <eos>"에 정답은 한 칸 밀린 "je suis professeur <eos> <pad>"입니다.
- **학습**: 손실이 3.79(에포크 1) → 0.46(50) → 0.06(100) → 0.02(200)로 줄었습니다.

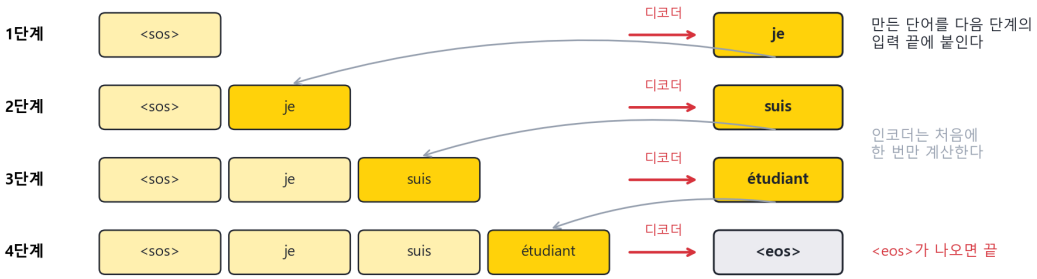



그림 14 학습에 없던 "i am a student"를 번역하는 과정(실습 7의 실제 출력). 만든 단어를 입력 끝에 붙여 가며 한 단어씩 만든다

**결과.** 학습에서 빼 둔 11문장을 모두 맞게 번역했습니다. "he is not ready"는 "il n'est pas prêt"로, "we are not teachers"는 "nous ne sommes pas professeurs"로 나왔습니다. 영어의 not 하나가 프랑스어에서는 동사의 앞뒤(ne … pas)로 갈라지고, 주어에 따라 동사와 서술어의 모양이 바뀌는 것을 맞췄습니다.

**읽을 때 주의할 점.** 시험 문장은 학습에 없던 조합이지만, 단어와 문장의 틀은 모두 학습에서 본 것입니다. "주어 + be 동사 + 서술어"라는 틀 안에서 새 조합을 만든 것이고, 틀 밖의 문장("i like students")은 번역하지 못합니다. 실제 번역기는 수백만 문장으로 학습합니다.

**인코더-디코더 어텐션 들여다보기.** "we are not at home" → "nous ne sommes pas à la maison"에서, 프랑스어 단어를 만들 때 영어의 어느 단어를 봤는지를 마지막 디코더 층에서 꺼냈습니다.

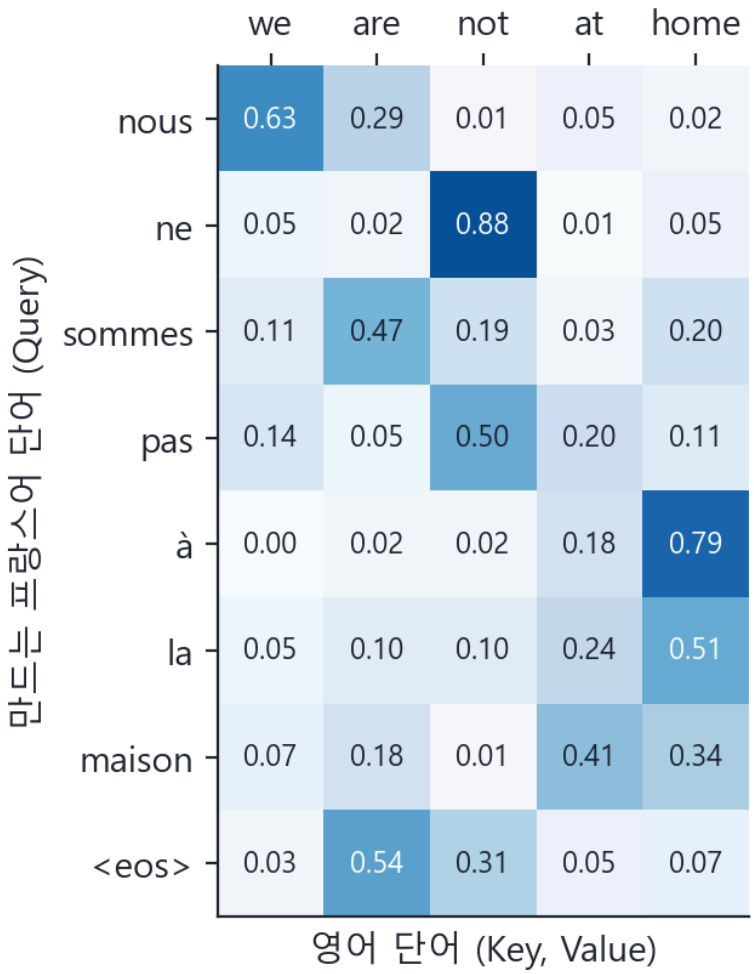



그림 15 행은 만드는 프랑스어 단어(Query), 열은 영어 단어(Key, Value). 칸의 값은 어텐션 분포(두 헤드의 평균)

- nous는 we(0.63)를, ne는 not(0.88)을, pas도 not(0.50)을 가장 많이 봅니다. 영어의 not 하나를 프랑스어의 두 단어가 각자 참고합니다.
- sommes는 are(0.47)를, à는 home(0.79)을 봅니다. "at home"과 "à la maison"은 단어 수가 달라 일대일로 짝지어지지 않고, à, la, maison이 at과 home에 나뉘어 걸칩니다.
- <eos> 행은 are(0.54), not(0.31)처럼 뜻으로 설명하기 어렵습니다. 어텐션 분포가 항상 사람이 생각하는 단어 짝과 일치하는 것은 아닙니다. 아무도 "ne는 not을 봐라"라고 가르치지 않았고, 번역을 맞히도록 학습하는 과정에서 이런 비율이 생겼습니다.

비유

### 가림판을 쓰는 통역 연습

통역 연습생이 모범 번역문을 가림판으로 덮고, 한 단어씩 가림판을 내리며 "다음에 올 단어"를 맞혀 봅니다. 뒤의 단어가 보이면 연습이 되지 않습니다. 단어를 고를 때마다 원문을 다시 훑어보며 지금 필요한 부분을 찾습니다. 실전에서는 모범 번역문이 없으므로, 자기가 방금 말한 단어에 이어서 다음 단어를 말합니다.

| | |
|---|---|
| 모범 번역문을 보며 연습 | = = 교사 강요 |
| 가림판 | = = 룩-어헤드 마스크 |
| 원문을 다시 훑어보기 | = = 인코더-디코더 어텐션 |
| 자기 말에 이어서 말하기 | = = 번역할 때의 한 단어씩 생성 |

실습 5

## 룩-어헤드 마스크를 만들고, 미래를 못 보는지 확인하기

```python
import torch
import torch.nn as nn

torch.manual_seed(0)
words = ["<sos>", "je", "suis", "étudiant"]
n = len(words)

# 1) 룩-어헤드 마스크: 자기보다 뒤에 있는 단어를 가린다 (True = 가릴 자리)
mask = torch.triu(torch.ones(n, n, dtype=torch.bool), diagonal=1)
print("룩-어헤드 마스크 (1 = 가림)")
print("          " + "".join(f"{w:>10}" for w in words))
for w, row in zip(words, mask):
    print(f"{w:>10}" + "".join(f"{int(v):>10}" for v in row.tolist()))

# 2) 스코어 행렬에 적용: 가릴 자리에 아주 작은 값을 넣고 소프트맥스
scores = torch.randn(n, n)                                  # 설명용 스코어
attn = torch.softmax(scores.masked_fill(mask, -1e9), dim=-1)
print("\n마스킹한 뒤의 어텐션 분포 (행마다 합 = 1)")
for w, row in zip(words, attn):
    print(f"{w:>10}" + "".join(f"{v:10.2f}" for v in row.tolist()))

# 3) 정말 미래를 못 보는가: 마지막 단어를 다른 것으로 바꿔 본다
mha = nn.MultiheadAttention(embed_dim=8, num_heads=2, batch_first=True)
X = torch.randn(1, n, 8)                                    # <sos> je suis étudiant
X2 = X.clone()
X2[0, 3] = torch.randn(8)                                   # 마지막 단어만 다른 단어로

out_masked, _ = mha(X, X, X, attn_mask=mask)
out_masked2, _ = mha(X2, X2, X2, attn_mask=mask)
out_open, _ = mha(X, X, X)
out_open2, _ = mha(X2, X2, X2)

same_masked = torch.allclose(out_masked[0, :3], out_masked2[0, :3], atol=1e-6)
same_open = torch.allclose(out_open[0, :3], out_open2[0, :3], atol=1e-6)
print("\n마지막 단어를 바꿨을 때, 앞 세 단어의 출력이 그대로인가?")
print("  마스크 있음:", same_masked, " (미래를 보지 못한다)")
print("  마스크 없음:", same_open, "(미래의 단어가 앞 단어의 출력에 섞인다)")

# 4) 파이토치가 만들어 주는 마스크: 가릴 자리가 -inf, 나머지가 0
print("\n파이토치가 만들어 주는 마스크")
print(nn.Transformer.generate_square_subsequent_mask(n))
```

실행 결과

```
룩-어헤드 마스크 (1 = 가림)
                 <sos>        je      suis  étudiant
     <sos>           0         1         1         1
        je           0         0         1         1
      suis           0         0         0         1
  étudiant           0         0         0         0

마스킹한 뒤의 어텐션 분포 (행마다 합 = 1)
     <sos>      1.00      0.00      0.00      0.00
        je      0.54      0.46      0.00      0.00
      suis      0.45      0.09      0.46      0.00
  étudiant      0.13      0.41      0.36      0.09

마지막 단어를 바꿨을 때, 앞 세 단어의 출력이 그대로인가?
  마스크 있음: True  (미래를 보지 못한다)
  마스크 없음: False (미래의 단어가 앞 단어의 출력에 섞인다)
```

```
파이토치가 만들어 주는 마스크
tensor([[0., -inf, -inf, -inf],
        [0., 0., -inf, -inf],
        [0., 0., 0., -inf],
        [0., 0., 0., 0.]])
```

직접 해 보기 `diagonal=1` 을 `diagonal=0` 으로 바꾸면 마스크가 어떻게 달라지나요? 그러면 첫 단어 <sos>는 무엇을 볼 수 있나요?

## 실습 6 부품을 조립한 작은 트랜스포머 (파일로 저장)

```python
# 지금까지 본 부품을 조립한 작은 트랜스포머. 코랩에서는 셀 맨 위에
# %%writefile mini_transformer.py  를 붙여 파일로 저장한 뒤,
# 다음 실습에서 import 한다.
import math

import torch
import torch.nn as nn


def positional_encoding(max_len, d_model):
    pos = torch.arange(max_len).float().unsqueeze(1)
    two_i = torch.arange(0, d_model, 2).float()
    angle = pos / 10000 ** (two_i / d_model)
    pe = torch.zeros(max_len, d_model)
    pe[:, 0::2], pe[:, 1::2] = torch.sin(angle), torch.cos(angle)
    return pe


class EncoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff):
        super().__init__()
        self.self_attn = nn.MultiheadAttention(d_model, num_heads, batch_first=True)
        self.ffnn = nn.Sequential(nn.Linear(d_model, d_ff), nn.ReLU(),
                                  nn.Linear(d_ff, d_model))
        self.norm1, self.norm2 = nn.LayerNorm(d_model), nn.LayerNorm(d_model)

    def forward(self, x, pad_mask):
        # 서브층 1: 셀프 어텐션 (Q = K = V = x) + Add & Norm
        a, _ = self.self_attn(x, x, x, key_padding_mask=pad_mask)
        x = self.norm1(x + a)
        # 서브층 2: FFNN + Add & Norm
        return self.norm2(x + self.ffnn(x))


class DecoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff):
        super().__init__()
        attention = lambda: nn.MultiheadAttention(d_model, num_heads, batch_first=True)
        self.self_attn, self.cross_attn = attention(), attention()
        self.ffnn = nn.Sequential(nn.Linear(d_model, d_ff), nn.ReLU(),
                                  nn.Linear(d_ff, d_model))
        self.norm1, self.norm2 = nn.LayerNorm(d_model), nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model)

    def forward(self, y, enc_out, look_ahead_mask, src_pad_mask):
        # 서브층 1: 마스크드 셀프 어텐션 (미래 단어를 가린다)
        a, _ = self.self_attn(y, y, y, attn_mask=look_ahead_mask)
        y = self.norm1(y + a)
        # 서브층 2: 인코더-디코더 어텐션 (Q = 디코더, K = V = 인코더의 출력)
        a, self.attn_weights = self.cross_attn(y, enc_out, enc_out,
                                               key_padding_mask=src_pad_mask)
        y = self.norm2(y + a)
        # 서브층 3: FFNN
        return self.norm3(y + self.ffnn(y))


class MiniTransformer(nn.Module):
    def __init__(self, src_vocab, tgt_vocab, d_model=32, num_heads=2, d_ff=64,
                 num_layers=2, max_len=20):
        super().__init__()
        self.d_model = d_model
        self.src_embed = nn.Embedding(src_vocab, d_model, padding_idx=0)
        self.tgt_embed = nn.Embedding(tgt_vocab, d_model, padding_idx=0)
        self.register_buffer("pe", positional_encoding(max_len, d_model))
        self.encoders = nn.ModuleList(
            [EncoderLayer(d_model, num_heads, d_ff) for _ in range(num_layers)])
        self.decoders = nn.ModuleList(
            [DecoderLayer(d_model, num_heads, d_ff) for _ in range(num_layers)])
        self.out = nn.Linear(d_model, tgt_vocab)     # 단어 집합 크기만큼의 점수

    def embed(self, layer, ids):
        # 임베딩에 √d_model 을 곱하고(논문의 방식) 포지셔널 인코딩을 더한다
        return layer(ids) * math.sqrt(self.d_model) + self.pe[:ids.size(1)]

    def encode(self, src):
```

```
        pad_mask = src == 0                          # 패딩 자리 = True
        x = self.embed(self.src_embed, src)
        for layer in self.encoders:
            x = layer(x, pad_mask)
        return x, pad_mask

    def decode(self, tgt, enc_out, src_pad_mask):
        n = tgt.size(1)
        look_ahead = torch.triu(torch.ones(n, n, dtype=torch.bool), diagonal=1)
        y = self.embed(self.tgt_embed, tgt)
        for layer in self.decoders:
            y = layer(y, enc_out, look_ahead, src_pad_mask)
        return self.out(y)                           # (배치, 길이, 단어 집합 크기)

    def forward(self, src, tgt):
        enc_out, pad_mask = self.encode(src)
        return self.decode(tgt, enc_out, pad_mask)
```

직접 해 보기 이 파일은 실행해도 출력이 없습니다. `EncoderLayer`와 `DecoderLayer`의 `forward`를 그림(인코더 층, 디코더 층)과 한 줄씩 맞춰 보세요. 잔차 연결은 코드의 어느 부분인가요?

## 실습 7 작은 트랜스포머로 영어를 프랑스어로 번역하기

```
# mini_transformer.py(앞 실습)를 먼저 저장해 둔다 (CPU로 약 1분)
import torch
import torch.nn as nn
from mini_transformer import MiniTransformer

torch.manual_seed(0)

# 1) 작은 번역 데이터: 주어 5개 × 서술어 9개 × (긍정, 부정) = 90문장
subjects = [("i am", "je suis", 0), ("you are", "tu es", 0), ("he is", "il est", 0),
            ("we are", "nous sommes", 1), ("they are", "ils sont", 1)]
# (영어 단수, 영어 복수, 프랑스어 단수, 프랑스어 복수)
predicates = [("a student", "students", "étudiant", "étudiants"),
              ("a teacher", "teachers", "professeur", "professeurs"),
              ("a doctor", "doctors", "médecin", "médecins"),
              ("happy", "happy", "content", "contents"),
              ("tired", "tired", "fatigué", "fatigués"),
              ("ready", "ready", "prêt", "prêts"),
              ("here", "here", "ici", "ici"),
              ("late", "late", "en retard", "en retard"),
              ("at home", "at home", "à la maison", "à la maison")]
pairs = []
for en_s, fr_s, plural in subjects:
    for pred in predicates:
        en_p, fr_p = pred[plural], pred[2 + plural]
        fr_subj, fr_verb = fr_s.split()
        pairs.append((f"{en_s} {en_p}", f"{fr_s} {fr_p}"))
        # 부정문: ne ... pas. 모음 앞에서는 ne가 n'로 줄어든다 (il n'est pas)
        neg = f"n'{fr_verb}" if fr_verb[0] == "e" else f"ne {fr_verb}"
        pairs.append((f"{en_s} not {en_p}", f"{fr_subj} {neg} pas {fr_p}"))

# 학습에서 빼 둘 문장 10개 (모델이 한 번도 보지 못한 조합)
test_idx = set(torch.randperm(len(pairs))[:10].tolist())
test_idx.add(pairs.index(("i am a student", "je suis étudiant")))
train = [p for i, p in enumerate(pairs) if i not in test_idx]
test = [p for i, p in enumerate(pairs) if i in test_idx]
print("전체", len(pairs), "문장 = 학습", len(train), "+ 시험", len(test))
print("예:", pairs[0], "/", pairs[1])

def build_vocab(sentences):
    words = sorted({w for s in sentences for w in s.split()})
    special = {"<pad>": 0, "<sos>": 1, "<eos>": 2}
    return {**special, **{w: i + 3 for i, w in enumerate(words)}}

src_vocab = build_vocab([en for en, _ in pairs])
tgt_vocab = build_vocab([fr for _, fr in pairs])
tgt_words = {i: w for w, i in tgt_vocab.items()}
print("영어 단어 집합:", len(src_vocab), " 프랑스어 단어 집합:", len(tgt_vocab))

def to_tensor(sentences, vocab, sos_eos=False):
    rows = [[vocab[w] for w in s.split()] for s in sentences]
    if sos_eos:
        rows = [[1] + r + [2] for r in rows]
    length = max(len(r) for r in rows)
    return torch.tensor([r + [0] * (length - len(r)) for r in rows])

src = to_tensor([en for en, _ in train], src_vocab)
tgt = to_tensor([fr for _, fr in train], tgt_vocab, sos_eos=True)
dec_in, dec_target = tgt[:, :-1], tgt[:, 1:]         # 교사 강요: 한 칸 밀린 정답
print("인코더 입력:", tuple(src.shape), " 디코더 입력:", tuple(dec_in.shape),
      " 정답:", tuple(dec_target.shape))
print("  디코더 입력 예:", " ".join(tgt_words[i] for i in dec_in[1].tolist()))
print("  그 정답       :", " ".join(tgt_words[i] for i in dec_target[1].tolist()))
```

```python
# 2) 모델과 학습
model = MiniTransformer(len(src_vocab), len(tgt_vocab))
out_shape = tuple(model(src, dec_in).shape)
print("모델 출력:", out_shape, "= (문장, 길이, 프랑스어 단어 수)")
criterion = nn.CrossEntropyLoss(ignore_index=0)      # 패딩 자리는 손실에서 뺀다
optimizer = torch.optim.Adam(model.parameters(), lr=0.002)
history = []
for epoch in range(1, 201):
    logits = model(src, dec_in)
    loss = criterion(logits.reshape(-1, len(tgt_vocab)), dec_target.reshape(-1))
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    history.append(loss.item())
    if epoch in (1, 50, 100, 200):
        print(f"에포크 {epoch:3d}: 손실 {loss.item():.4f}")

# 3) 번역: <sos>에서 시작해 한 단어씩 만들고, 만든 단어를 다시 입력에 붙인다
def translate(sentence, show=False):
    model.eval()
    with torch.no_grad():
        enc_out, pad_mask = model.encode(to_tensor([sentence], src_vocab))
        ids = [1]                                    # <sos>
        for step in range(10):
            logits = model.decode(torch.tensor([ids]), enc_out, pad_mask)
            next_id = logits[0, -1].argmax().item()  # 마지막 자리의 예측만 쓴다
            if show:
                now = " ".join(tgt_words[i] for i in ids)
                word = tgt_words[next_id]
                print(f"  {step + 1}단계: 입력 [{now}] → 다음 단어 '{word}'")
            if next_id == 2:                         # <eos>가 나오면 끝
                break
            ids.append(next_id)
    return " ".join(tgt_words[i] for i in ids[1:])

print("\n'i am a student'를 번역하는 과정 (학습에 없던 문장)")
translate("i am a student", show=True)

correct = 0
print("\n학습에 없던 문장 번역")
for en, fr in test:
    pred = translate(en)
    correct += pred == fr
    mark = "O" if pred == fr else f"X (정답: {fr})"
    print(f"  {en:22s} → {pred:31s} {mark}")
print(f"시험 문장 {len(test)}개 중 {correct}개 정답")

# 4) 인코더-디코더 어텐션: 프랑스어 단어를 만들 때 영어의 어느 단어를 봤는가
en, fr = "we are not at home", "nous ne sommes pas à la maison"
enc_out, pad_mask = model.encode(to_tensor([en], src_vocab))
model.decode(to_tensor([fr], tgt_vocab, sos_eos=True)[:, :-1], enc_out, pad_mask)
weights = model.decoders[-1].attn_weights[0]         # (프랑스어 길이, 영어 길이)
print("\n어텐션 분포:", tuple(weights.shape), "(행 = 만들 단어, 열 = 영어 단어)")
print("  " * 10 + "".join(f"{w:>7}" for w in en.split()))
for w, r in zip(fr.split() + ["<eos>"], weights):
    print(f"{w:>10}" + "".join(f"{v:7.2f}" for v in r.tolist()))
torch.save({"history": history, "weights": weights.detach(), "en": en.split(),
            "fr": fr.split() + ["<eos>"]}, "translate_result.pt")
```

실행 결과

```
전체 90 문장 = 학습 79 + 시험 11
예: ('i am a student', 'je suis étudiant') / ('i am not a student', 'je ne suis pas étudiant')
영어 단어 집합: 26  프랑스어 단어 집합: 35
인코더 입력: (79, 5)  디코더 입력: (79, 8)  정답: (79, 8)
  디코더 입력 예: <sos> je suis professeur <eos> <pad> <pad> <pad>
  그 정답       : je suis professeur <eos> <pad> <pad> <pad> <pad>
모델 출력: (79, 8, 35) = (문장, 길이, 프랑스어 단어 수)
에포크   1: 손실 3.7905
에포크  50: 손실 0.4611
에포크 100: 손실 0.0600
에포크 200: 손실 0.0152

'i am a student'를 번역하는 과정 (학습에 없던 문장)
  1단계: 입력 [<sos>] → 다음 단어 'je'
  2단계: 입력 [<sos> je] → 다음 단어 'suis'
  3단계: 입력 [<sos> je suis] → 다음 단어 'étudiant'
  4단계: 입력 [<sos> je suis étudiant] → 다음 단어 '<eos>'

학습에 없던 문장 번역
  i am a student         → je suis étudiant                O
  i am not a teacher     → je ne suis pas professeur       O
  i am happy             → je suis content                 O
```

```
  he is not a teacher    → il n'est pas professeur         O
  he is tired            → il est fatigué                  O
  he is not ready        → il n'est pas prêt               O
  we are not teachers    → nous ne sommes pas professeurs  O
  we are not happy       → nous ne sommes pas contents     O
  we are at home         → nous sommes à la maison         O
  they are students      → ils sont étudiants              O
  they are not ready     → ils ne sont pas prêts           O
시험 문장 11개 중 11개 정답

어텐션 분포: (8, 5) (행 = 만들 단어, 열 = 영어 단어)
               we    are    not     at   home
      nous   0.63   0.29   0.01   0.05   0.02
        ne   0.05   0.02   0.88   0.01   0.05
    sommes   0.11   0.47   0.19   0.03   0.20
       pas   0.14   0.05   0.50   0.20   0.11
         à   0.00   0.02   0.02   0.18   0.79
        la   0.05   0.10   0.10   0.24   0.51
    maison   0.07   0.18   0.01   0.41   0.34
     <eos>   0.03   0.54   0.31   0.05   0.07
```

직접 해 보기 `translate("i like students")` 를 실행하면 오류가 납니다. 왜일까요? 학습에 있던 단어만 쓰되 틀이 다른 `translate("happy we are")` 는 어떻게 번역하나요?

확인 문제

## ⑥ 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

**1** 룩-어헤드 마스크는 각 단어가 자기보다 앞에 있는 단어를 보지 못하게 한다. (O / X) 이름 _____

**2** 인코더-디코더 어텐션은 셀프 어텐션이다. (O / X) 이름 _____

**3** 인코더-디코더 어텐션에서 Key와 Value는 어디에서 오나요? 이름 _____

① 인코더의 마지막 층의 출력 ② 디코더의 입력 ③ 포지셔널 인코딩

**4** 번역할 때 디코더가 멈추는 조건은? 이름 _____

① <eos>가 나올 때 ② <sos>가 나올 때 ③ 입력 문장과 길이가 같아질 때

**5** 디코더 층의 서브층은 모두 _____개다. 이름 _____

**6** 학습할 때 디코더에 정답 문장을 입력으로 넣어 주는 방법을 _____ 강요라고 한다. 이름 _____

**7** 룩-어헤드 마스크가 왜 필요한지, 가림판을 쓰는 통역 연습에 빗대어 설명해 보세요. 이름 _____

원문: 18-01 트랜스포머 완전 정복 — https://wikidocs.net/217018 (교재의 한 소챕터를 여섯 부분으로 나눴습니다. 교재에는 실습 코드가 없어 실습 1~7은 이 자료에서 만들었고, 실습과 그림의 벡터는 4칸, 2칸으로 줄였습니다. 추론 과정, 학습과 추론의 비교, $\sqrt{d_k}$와 잔차 연결의 이유 설명은 교재에 없는 보충입니다)

# 오늘의 요약

다시 보기




정의 모음

| 개념 | 정의 | 예시 |
|---|---|---|
| 트랜스포머 | 어텐션만으로 만든 인코더-디코더 모델 | 인코더 6층, 디코더 6층 |
| 포지셔널 인코딩 | 위치마다 정해진 사인·코사인 값을 임베딩에 더함 | pos 1: [0.841, 0.540, 0.010, 1.000] |
| 셀프 어텐션 | Q, K, V가 같은 문장에서 나오는 어텐션 | it이 animal을 참고 |
| 스케일드 닷-프로덕트 어텐션 | $\mathrm{softmax}(QK^T/\sqrt{d_k})V$ | I의 분포 [0.45, 0.00, 0.53, 0.02] |
| 멀티 헤드 어텐션 | 작은 어텐션 여러 개를 동시에 하고 이어 붙임 | (4, 2) 두 개 → (4, 4) |
| 패딩 마스크 | Key가 <PAD>인 열을 가림 | [0.63, 0.26, 0.11, 0.00] |
| 포지션-와이즈 FFNN | 단어마다 같은 신경망을 따로 적용 | 512 → 2048 → 512 |
| Add & Norm | $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$ | [1, 2, 3, 6] → [-1.07, -0.53, 0.00, 1.60] |
| 룩-어헤드 마스크 | 자기보다 뒤의 단어를 가림 | je는 <sos>, je만 봄 |
| 인코더-디코더 어텐션 | Q는 디코더, K와 V는 인코더의 출력 | ne가 not을 0.88 참고 |

다음

### 다음 자료 예고: 19장 BERT (10/16)

트랜스포머를 반으로 나눠 쓰는 모델들이 나옵니다. **BERT**는 인코더만 쌓아 문장을 이해하는 데 쓰고, **GPT**는 디코더만 쌓아 글을 이어 쓰는 데 씁니다. 오늘 본 셀프 어텐션, 마스크, Add & Norm이 그대로 들어 있으므로, 달라지는 것은 "무엇을 맞히도록 학습시키는가"입니다.

# 정답과 해설

### ① 큰 그림

1. X — 인코더-디코더 구조는 그대로 두고, 안의 RNN을 어텐션으로 바꿨습니다.
2. O
3. ②
4. ①
5. 512
6. $d_{model}$
7. 예시 답: 긴 문장에서 앞 단어의 정보가 여러 셀을 지나며 흐려지고, 차례로 계산해야 해서 느리다. 귓속말 전달처럼 옆 사람에게만 넘기면 말이 바뀌고 오래 걸리지만, 원탁 회의처럼 모두가 서로를 직접 들으면 한 번에 닿고 동시에 할 수 있다. 트랜스포머는 어텐션으로 원탁 회의를 만든 것이다.

### ② 포지셔널 인코딩

1. X — 이어 붙이지 않고 더합니다. 모양은 그대로입니다.
2. X — 사인과 코사인으로 정해진 값이고 학습되지 않습니다.
3. ①
4. ② — 칸 번호는 $2i$, $2i+1$입니다.
5. 코사인
6. -1
7. 예시 답: 포지셔널 인코딩은 단어의 위치를 나타내는 벡터로, 임베딩 벡터에 더해 모델에 넣는다. 원고를 한꺼번에 펼쳐 놓으면 순서를 알 수 없어 장마다 쪽 번호를 찍듯이, 단어를 한꺼번에 읽는 트랜스포머에게 단어마다 위치 값을 더해 순서를 알려 준다.

### ③ 셀프 어텐션

1. O — 출처가 같다는 뜻이고, 서로 다른 가중치 행렬을 곱해 값은 다릅니다.
2. O — 소프트맥스를 행마다 씌웠기 때문입니다.
3. ①
4. ② — (단어 수, 단어 수)
5. 64
6. V (값, Value)
7. 예시 답: 내 질문(Q)을 책등의 제목(K)과 맞춰 보며 점수를 매기고(내적), 점수가 너무 벌어지지 않게 줄인 뒤(스케일), 책마다 읽을 비중을 합이 1이 되게 정하고(소프트맥스), 그 비중대로 책의 내용(V)을 섞어 노트를 만든다(가중합).

### ④ 멀티 헤드 어텐션과 패딩 마스크

1. X — 헤드마다 가중치가 다릅니다. 그래서 서로 다른 관계를 봅니다.
2. X — 이어 붙이면 다시 (seq_len, $d_{model}$)이 됩니다.
3. ①
4. ①
5. 전 — 그래야 남은 단어들끼리 합이 1이 됩니다.
6. 64
7. 예시 답: 단어 사이의 관계는 여러 가지인데 어텐션 한 번은 비율 하나만 만든다. 노란 펜으로 "누가", 초록 펜으로 "무엇을"을 따로 표시하듯 헤드마다 다른 관계를 보게 하고, 마지막에 표시를 한 장에 모으듯 결과를 이어 붙인다.

### ⑤ 인코더 층 완성

1. X — 단어마다 따로 적용합니다. 정보를 주고받는 곳은 어텐션입니다.
2. O
3. ①
4. ①
5. 2048

6. 잔차
7. 예시 답: 원문은 그대로 두고 고칠 내용만 메모로 덧붙이면 여러 사람을 거쳐도 원래 내용이 남는다. 잔차 연결은 입력을 출력에 더해 원래 정보와 기울기가 지나갈 길을 남기고, 층 정규화는 넘기기 전에 정서하듯 값의 크기를 일정하게 맞춰 학습을 안정시킨다.

### ⑥ 디코더

1. X — 뒤에 있는 단어를 보지 못하게 합니다.
2. X — Query는 디코더, Key와 Value는 인코더에서 와서 출처가 다릅니다.
3. ①
4. ①
5. 3 — 마스크드 셀프 어텐션, 인코더-디코더 어텐션, FFNN
6. 교사
7. 예시 답: 디코더는 정답 문장을 한꺼번에 받으므로, 가리지 않으면 다음 단어를 옆에서 보고 쓸 수 있어 아무것도 배우지 못한다. 모범 번역문을 가림판으로 덮고 다음 단어를 맞혀 보는 연습처럼, 룩-어헤드 마스크로 뒤의 단어를 가려 실제 번역할 때와 같은 조건을 만든다.

# 용어 정리

| 용어 | 정의 |
|---|---|
| 트랜스포머 | RNN 없이 어텐션만으로 인코더와 디코더를 만든 모델 |
| $d_{model}$ | 단어 벡터의 크기이자 모든 층의 입출력 크기 (논문 512) |
| num_layers / num_heads / $d_{ff}$ | 쌓는 층의 수 (6) / 어텐션 헤드의 수 (8) / FFNN의 은닉층 크기 (2048) |
| 포지셔널 인코딩 | 단어의 위치를 나타내는 벡터. 임베딩에 더한다 |
| 쿼리 / 키 / 값 | 질문하는 쪽 / 질문과 맞춰 보는 이름표 / 가져올 내용 |
| 셀프 어텐션 | Q, K, V가 같은 문장에서 나오는 어텐션 |
| 어텐션 스코어 / 분포 / 값 | Q와 K의 점수 / 소프트맥스로 만든 합 1의 비율 / 그 비율로 V를 가중합한 결과 |
| 스케일드 닷-프로덕트 어텐션 | 내적을 $\sqrt{d_k}$로 나눠 점수를 내는 어텐션 |
| 멀티 헤드 어텐션 | 어텐션 여러 개를 서로 다른 가중치로 동시에 하고 이어 붙이는 것 |
| 마스킹 | 어텐션에서 특정 위치를 참고하지 못하게 가리는 것 |
| 패딩 마스크 / 룩-어헤드 마스크 | <PAD>를 가림 / 자기보다 뒤의 단어를 가림 |
| 포지션-와이즈 FFNN | 단어 자리마다 같은 완전 연결 신경망을 따로 적용하는 서브층 |
| 잔차 연결 | 서브층의 입력을 출력에 더하는 것 |
| 층 정규화 | 단어 벡터 하나 안에서 평균 0, 분산 1로 맞추는 것 |
| 인코더-디코더 어텐션 | Query는 디코더, Key와 Value는 인코더의 출력인 어텐션 |
| 교사 강요 | 학습할 때 디코더에 정답 문장을 입력으로 넣어 주는 방법 |